State Is All You Need

——以中型 byte 级有状态语言模型 Gridman/Ouro 为对象的实证研究（初稿）

作者列表 [待补：待填]

单位 / 通讯作者 [待补：待填]

2026 年 10 月 23 日

本文状态（草稿，投稿前请先读这一段）

- 所有数字来自**单一检查点**：`gridman_medium_v_0d3_pretrain`，step 144000/705600，即训练进度**约 20%**。模型行为描述以现行实现 **v__0d4** 为准（memory 改为序列级私有槽位，两版存档不兼容；batch = 1 推理两版逐位对应，见 §3 表 1）。因此本文只做**条件间的相对比较**，任何绝对值（如 1.08 bit/byte）都**不可**与外部模型对照。加验证集、训完 medium、补同预算基线（§6.3）之前，本稿不可当作最终结果。
- 已完成并可报：A1/A2/A3/A5/A6（第 5 章）、B1-lite/B2/B2-freeze（第 6 章）、以及稳定性与状态性质（附录 D、E）。
- **未完成**：A4（状态错配）、B3（等效滑窗基线）、B4（needle 预实验）、F 线（重训类）。正文中未完成处一律以 [待补：] 标出，不预留空节。
- **已作废**：受控遗忘曲线（C1）因实现缺陷（答案位置整体偏移，实际在测补白字节）作废，不从正文引用；完整说明见附录 D.1。

摘要

标准语言模型把历史信息的载体固定在上下文窗口里：窗口内的 token 可被注意力直接访问，窗口之外的历史要么不可见，要么必须以随长度增长的成本重新写回窗口。持续状态是另一种载体——一段形状固定的张量，跨上下文持续存在，在前向过程中被反复读写，其容量**不随历史长度增长**。本文把「固定大小的持续状态能承载多少跨上下文信息」作为可以用实验正面验证的经验问题，在一个 437.8M 参数的中型 byte 级有状态语言模型（Gridman/Ouro medium）上做系统测量。评测只使用训练到 20%（step 144000/705600）的单一检查点，全程 teacher-forced、不做任何重训，并把操纵旋钮全部落在架构暴露的状态接口上：**起点状态**（热启动 / 空白）、**读写路径**（commit / lock）、**组件手术**（重置 / 冻结）。

主要结果：（1）状态的跨上下文携带**成立**：在同一计分窗口下，热启动起点比失忆起点低 3.73 bit/byte（1.59 对 5.32），且该差距在窗口内不收敛；锁定跨 patch 写入的代价是 $+1.00 \pm 0.09$ bit/byte，并在上下文增长时**累积**（后段 0.77 对前段 0.58 nat）；跨文档边界重置造成 +0.45 nat 的冲击，但中位数 1 个 patch 即自愈。（2）组件归因给出一个反直觉的结论：自然文本上的跨 patch 适应**几乎全部由块级循环状态（recurrent c_state）承担**——冻结它等价于冻结全部写入（+1.02 对 +1.03 bit/byte），而冻结 delta 规则矩阵记忆（memory）的写入代价 ≈ 0（-0.0005 bit/byte）；矩阵记忆的价值不在增量适应，而在**读取训练稳态先验**。（3）有效上下文**有限**：自然重复带来的复现增益集中在 1k token 以内（+0.084 nat），4k 之外衰减到零甚至转负；冻结 recurrent 会精确抹平该增益。这修正了「固定状态长程记忆 $\geq$ 256K 不衰减」的初步结论（该结论源于一次分桶实现的缺陷，已作废重测）。（4）永续性侧的证据是正面的：96KB / 512KB 连续写入下状态范数有界缓增（memory $\times 1.02/\times 1.13$、MTM $\times 1.02/\times 1.07$，STM/queue 恒定），重置频率扫描单调（512tok $\to \infty$：1.215 $\to$ 1.039 bit/byte），未见噪声累积。（5）一条必须写进评测协议的警示：热启动状态里含有**逐样本的训练痕迹**——在训练消费区文本上热启动的优势比未消费文本低 0.35 ± 0.20 bit/byte，且这一成分几乎全部由状态承载（去掉热启动后该差距消失：$+0.36 \pm 0.83$），因此主表必须**双报起步状态**。

结论：固定大小的持续状态在这个规模上确实承载了可测的跨上下文信息与稳定的长时写入能力，但其**有效记忆距离约为 1-4k token**，与「永续上下文」的强主张尚有量级差距；且承载者主要是循环状态而非参数化矩阵记忆。本文同时给出未完成项（状态错配、等效滑窗基线、精确检索预实验）与已作废实验（受控遗忘曲线）的完整说明。

关键词：有状态语言模型；持续状态；固定大小记忆；delta 规则快权重；有效上下文；teacher-forced bits/byte；评测协议

Abstract

Standard language models carry history in the context window: tokens inside the window are directly attended to, while history outside it is either invisible or must be replayed at a cost that grows with length. A *persistent state* is an alternative carrier—a fixed-shape tensor that exists across contexts, is read and written at every forward pass, and whose capacity does not grow with history length. We turn "how much cross-context information can a fixed-size persistent state carry" into an empirical question and measure it on a 437.8M-parameter byte-level stateful LM (Gridman/Ouro medium), using a single checkpoint at 20% of training (step 144000/705600), teacher-forced evaluation, and no retraining. All control knobs are architecture-exposed state operations: the *starting state* (hot start vs. blank), the *read/write path* (commit vs. lock), and *component surgery* (reset / freeze). We find that (i) cross-context carriage holds—on a common scoring window the hot start beats a blank start by 3.73 bits/byte (1.59 vs. 5.32) with no convergence trend, and locking cross-patch writes costs $+1.00 \pm 0.09$ bits/byte, growing with context length; (ii) cross-patch adaptation is almost entirely carried by the block-level recurrent state—freezing it costs as much as locking all writes (+1.02 vs. +1.03 bits/byte), while freezing the global delta-rule matrix memory is free (-0.0005); (iii) the effective repetition memory is limited to about 1–4k tokens; and (iv) the hot-start state contains sample-level training traces, so any main table must report both starting states.

Keywords: stateful language model; persistent state; fixed-size memory; delta-rule fast weights; effective context; teacher-forced bits/byte

1 绪论

1.1 背景：语言模型的上下文与状态

先说循环网络这一路。LSTM/GRU 及其变体 [7, 13] 是最早把「固定大小的状态」写进架构的一路：一步一递归，历史被压缩进一个形状固定的隐状态。这个想法本身是对的，但它没有走到今天，原因有两条。其一，**状态太小**：一般只有 KB 到 MB 量级，容量撑不住长历史；其二，**效果不好**：长程依赖学不动、训练也不稳定，于是被「更大的窗口 + 注意力」一路取代。所以要把问题评准——症结不在「固定状态」这个想法，而在**当时的状态不够大、写入方式不够好**。本文接下来做的事，就是把状态放大：我们研究的模型（Gridman/Ouro medium）仅 delta 规则矩阵记忆一项就有 12,954,624 个 fp32 元素（约 49.4 MiB），推理时 batch= 1 的四件套状态切片合计 14,072,832 个 fp32 元素（约 53.7 MiB：memory 49.4 + MTM 4.0 + queue 0.25 + recurrent 0.016），比典型 RNN 状态高出两个到四个数量级。

既有架构只有一种信息载体：上下文窗口。每一步预测都通过注意力读取窗口内所有 token；窗口之外的历史模型完全看不见——除非把它重新写回窗口（重放），而重放的成本随长度增长（注意力计算与显存都随长度线性膨胀）[8, 20, 25]。窗口越开越大是一条路 [3, 9]，但它改变的是「一次能看多少」，不是「能带多少」。

持续状态是另一种载体。一段形状固定的张量，跨上下文持续存在，在每次前向中被读写，其容量**不随历史长度增长**：喂给它 1 千字节还是 100 万字节，状态大小不变。窗口用「更大」换「记得更多」，状态用「固定」换「持续」——两者的张力正是本文的问题：固定大小的状态到底能承担多少跨上下文的信息传递？这是一个可以用实验正面验证、给出定量结论的经验问题，而不是「架构看起来很优雅」式的主张。

全文围绕两点： **把状态做大**（scaling up）；**大状态带来的持续学习**——状态跨上下文持续被写入，模型在「读」的过程中仍在改变自己的行为，而不是读完即定。逻辑链全文固定为：*以前 RNN 效果不好 → 本文把状态放大 → 更大的状态带来持续学习。*

1.2 问题：固定大小的持续状态能否承载跨上下文信息

形式化。一次前向写成

$$\text{logits},\ s' = f(x, s; lock), \tag{1}$$

其中 x 是当前输入（本文实现中为一个 64 字节的 patch），s 是状态——**形状固定，与已处理的历史长度无关**，这就是「固定大小」的字面含义；s' 是写入后的新状态；$lock$ 是布尔开关，决定这一步 s 是否接受新写入（$lock = \text{true}$ 时状态只读，见 §3.3）。上下文的延续由 $s' = f(x, s)$ 在 patch 之间递归传递，不依赖 KV cache 的累积。

两个待验证的子问题。它们是本文的两条主张，第 5、6 章分别正面验证：

- **S1：状态是否持续携带跨上下文信息？** *成立条件：*热启动显著优于空白、正常写入显著优于 $lock$。只有当前者成立，才说明状态里带着信息；只有当后者成立，才说明「往状态里写」这件事有因果贡献。两条同时满足，S1 成立。
  - **热启动（hot start）：**评测起点用训练收敛时保存的稳态状态快照（§3.4），相当于「模型带着训练结束时的记忆来考试」。

  - **空白（blank）**：评测起点用全初始状态（恒等 memory + 零 recurrent/queue/MTM），相当于「模型失忆后开始读」。
  - *lock*：前向过程中状态只读不写——这段上下文的信息**不许**进入状态；它与「正常（commit）」构成一对受控对照，隔离出「写入」的因果贡献。

- **S2：固定状态的有效上下文窗口有多大？** *成立条件：* **有效窗口显著大于 0**——固定状态在损失上确实等效于一段非零的滑动窗口。**有效窗口**不是架构参数而是测出来的等效值 [待补：测量方法见 §6.3，需滑窗基线]。

S1 问「有没有」（定性），S2 问「有多少」（定量）。第 7 章的汇总表把这两问与第 5、6 章的判定句直接对上。

1.3 本文贡献

主线句：以前 RNN 效果不好，是因为状态太小；本文把状态放大，大状态让「跨上下文携带信息」成立，并进一步带来持续学习。三条贡献分别是这条链上的三段证据。

贡献一：起点状态的系统消融结论（回答 S1，第 5 章）。五组对照——热启动 vs 空白（A1）、写入 vs *lock*（A2）、跨文档边界连续 vs 重置（A3）、状态错配（A4）、组件级归因（A5/A6）——每组预先写明「状态在携带信息时结果应该长这样」的成立预期，合起来逐条判定状态是否真在携带信息。这是「状态放大了，但里面到底有没有东西」的直接回答。本文在该簇上的实测结论是：S1 **成立**，且携带者主要是块级循环状态（§5.5）。

贡献二：固定大小状态的有效窗口测量（回答 S2，第 6 章）。三把尺交叉定位：位置衰减（长文档上损失随位置怎么走，B1）、重复距离（自然重复内容隔多远还记着，B2）、等效滑窗（把状态折算成一个窗口宽度数字，B3）。本文已完成前两把尺，第三把尺与精确检索预实验（B4）未完成，如实标注。结论是**有效记忆距离约 1-4k token**：S2 的「显著大于 0」成立，但远未达到「永续上下文」量级。

贡献三：持续状态与有状态架构（第 3 章）。架构层面的贡献有两层含义，这里给出的是**状态在前**的表述： **状态是微观实现、可以显式调整**——它可以整体读出、冻结（*lock*）、置换（重置/换入其他状态），所以评测旋钮直接落在架构接口上，而不是像无状态模型那样只能从外部改提示词、改窗口； **记忆是宏观体现、可以多方面量度**——记忆不是外挂模块，就是四件套状态（memory / recurrent / queue / MTM，见 §3.2）在跨上下文尺度上表现出来的能力；**持续学习是它在时间维度上的表现**——状态持续被写入，行为随之改变。

说明：原大纲把「teacher-forced bits/byte 评测协议」列为贡献之一，本稿按要求删除——协议是支撑贡献一、二的手段，不单列为贡献；协议本身在第 4 章完整定义。

本文材料边界。本文所报实验全部基于**单一检查点**（step 144000/705600，约 20% 训练进度）的推理评测，不做任何重训；已完成 A1–A3、A5、A6、B2 及其组件归因、外加稳定性（E 线）、热启动记忆的领域分解与成员推断（D 线）、前向学习（G 线）；未完成 A4、B3、B4、F 线（需重训）；受控遗忘曲线（C1）因实现缺陷作废，完整说明见附录 D.1。

2 相关工作

2.1 持续状态架构：状态与记忆

本节把状态与记忆分开讲，因为两者是**实现与体现**的关系，不是并列的两个东西：**状态实现记忆**——状态是前向过程中每一步实际读写的张量对象（实现层）；**记忆是状态的宏观体现**——站在跨上下文的尺度看，「模型记得之前读过什么」这个能力，就是状态被反复读写的累积效果。所以 §2.1 先讲状态（微观、实现），再讲记忆（宏观、能力）。

2.1.1 状态：架构的实现层

定义。状态是模型内部一组形状固定的张量对象，随前向逐步读写，不随历史长度增长。

与 RNN 的关系：思想类似，实现差异很大。两者都用固定大小的循环状态承载历史，但本文研究的架构在四点上都不同于 RNN：（i）不是单一隐向量的逐步递归，而是**四件套按时间尺度分工**（delta 规则矩阵 memory + 逐层 c_state + 滚动 queue + 中期 MTM，见 §3.2）；（ii）memory 的写入用 **delta 规则快权重** [21, 22, 24, 26]，而不是 RNN 的门控线性更新；（iii）**规模不同**：RNN 的状态一般只有 KB–MB 级，本文的状态高出若干数量级（§1.1）——「把容量做大」在参数规模上已有系统性的经验规律 [15]，把同样的思路用在**状态**上正是本文的出发点；（iv）**训练方式不同**：RNN/RWKV 训练时状态从零初始化、一批读完即清零，单个状态最多跨一个样本（例如 2000 token）；本文的模型不按批次清空，上一批结束时的状态直接续进下一批，整条训练流是一条全局连续的流（§3.4）。最后、也是最关键的一点：状态被设计成**可以整体读出、冻结、置换的显式对象**——RNN 的隐状态没有对评测暴露这样的接口。

与测试时训练/记忆模块一类工作的关系。把「记忆」做成层内的可学习结构并非新想法：快权重 [1, 22]、TTT[23]、Titans[2] 都让模型在前向中写入某种内部状态；线性注意力/状态空间模型 [12, 16, 19] 则把递归写成可并行扫描的形式。本文研究的架构在「读取用 delta 规则、写入用误差驱动外积」这一点上与它们同源，区别在把该规则**嵌入到每一处线性投影**而不是只放在注意力位置，并把状态拆成四个时间尺度、以显式对象的形式暴露给评测。本文的实验关心的不是「谁更强」，而是**这个状态里到底有什么、能带多远**。

2.1.2 记忆：状态的宏观体现

定义。记忆 = 状态在跨上下文信息传递上表现出的宏观能力；它与 §2.1.1 的状态是同一个东西的两个观察尺度，**不是并列的两个模块**。

与记忆增强路线的对照。既有工作大多把记忆放在架构之外：外部记忆库或检索式增强 [11, 17] 从外部语料/存储里捞回信息；滑窗或长窗口 [3, 9] 把记忆放进更长的窗口；循环记忆 token[5] 把记忆放在一串可训练槽里并通过反向传播更新。这些路线的共同点是记忆**不在模型状态里**——要么在外存，要么在窗口，要么在参数。本文研究的记忆完全由持续状态承载：不依赖重放、不借助外部存储、不靠额外参数。

从「记忆全部落在持续状态上」到持续学习。记忆既然全部落在持续状态上，而状态在每次前向都被写入，那么模型在阅读过程中仍在改变自己的行为——这就是持续学习：学习不只发生在训练时，也发生在推理的前向之中（与上下文内学习这一现象相关但并不相同：ICL 是「看示例就会」[4, 6, 18]，本文关心的是「读着读着状态变了、行为随之变了」）。这与「损失可塑性」[10]

是同一个问题的两个方向：既有工作关心**反复训练时会不会变坏**，本文关心**一次前向流之内，行为会不会变好、变多少**。可验证的含义是：在分布内/分布外的文本流上，模型的有效行为会随前向推进而改变——本文用 A2/A6（写入的因果贡献与组件归因）、G 线（合成任务上的学习曲线）以及 D 线（热启动记忆的领域特异性）三组证据来量这个「读着读着变了多少」。

2.2 长上下文评测方法

needle（needle-in-a-haystack）[14]：在长文本中埋入一条「针」，读完后提问，测模型能否精确检索。它是二值的（对/错），测的是检索能力。本文只把它作为边界预实验（§6.4）——固定大小状态的强项未必是精确定位，所以如实限定 claim 边界，失败也写进 Limitations。

滑窗基线：用一个固定窗口宽度 W 的滑动窗口模型作参照，看固定状态的逐位置损失落在哪个 W 上，从而把「状态记了多少」折算成一个直观数字「等效窗口宽度」（§6.3）。

per-position 分析：不只报一个总分，而是把损失**按位置逐点对齐**报告随位置的走向。总分会掩盖结构：衰减形状、边界冲击、恢复长度只有逐位置曲线才看得见。本文的位置衰减曲线（§6.1）与边界恢复曲线（§5.3）都属于这一口径。

既有方法与本文的接点。既有方法测的是「上下文（窗口）能看多远」，本文测的是「固定状态能带多远」；两者底层指标口径相同（teacher-forced 逐位置损失），因此结果可以互相对照——这也意味着本文的评测协议必须把「起点状态」写清楚（§4.3），否则对照不成立。

2.3 创新点总结

vs RNN（§2.1.1）：思想类似——都用固定大小的循环状态承载历史；但实现差异很大——不是单一隐向量的逐步递归，而是四件套按时间尺度分工，memory 用 delta 规则快权重写入；规模上 RNN 状态只有 KB–MB 级，本文的状态高出若干数量级；训练方式上 RNN/RWKV 每批清零、本文状态跨批次连续（§3.4）；最关键的差别是状态被设计成**可整体读出、冻结、置换的显式对象**。

vs 记忆增强路线 / 长上下文（§2.1.2）：既有工作把记忆放在架构之外——外部记忆库、检索增强、或单纯堆长上下文窗口；本文的记忆完全由持续状态承载，不重放、不外挂，并在持续写入中得到持续学习。一句话：别人的记忆在状态之外，本文的记忆就是状态在跨上下文尺度上的宏观体现。

vs 既有评测方法（§2.2）：needle、滑窗基线、per-position 各自测的是「窗口能看多远」；本文沿用同一底层口径（teacher-forced 逐位置损失，结果可相互对照），但把操纵旋钮从「改窗口」换成「动状态」——热启动 / *lock* / 重置 / 冻结全部落在状态接口上。

vs 传统无状态架构：状态即 KV cache，随输入即生即灭、不可操纵，推理期不会因为读了新内容而改变自己；本文的状态是显式对象，可整体读出、冻结、置换，且持续写入带来持续学习。这正对应贡献三的第一层表述——**状态是微观实现，可以显式调整**。

汇总（围绕核心两点）：本文的创新点 = **状态放大**（scaling up，状态参数量相对 RNN 高出若干数量级，容量差两个到四个数量级）；**持续学习**（状态跨上下文持续写入，模型在前向中仍在改变自己的行为）。这两点的可评测性由架构保证——状态是显式对象（可动），记忆可由位置衰减 / 重复距离 / 等效窗口三把尺量（可测）。

3 模型：持续状态架构

3.1 整体结构（byte 级语言模型 + 有状态核心）

模型由两部分组成。

（1）byte 级语言模型。分词单位是**字节**而不是词：ByteTokenizer 直接以 UTF-8 字节为 token，词表大小 300（256 个字节值 + pad=256 + eos=257）。好处是不存在词表外的「词」，任何文本都能无损进出；损失以 **bits/byte** 计，跨语言、跨编码格式可比——这就是主指标口径的来源（§4.2）。

（2）有状态核心。一次前向吃一个 64 字节的 patch（记 $B = 64$）：

$$\text{logits},\ \text{state} = \text{model}(x,\ \text{state},\ \textit{lock_mem}), \qquad x \in \mathbb{Z}_B^{1\times 64}, \tag{2}$$

上下文的延续不靠 KV cache 累积，靠 $s' = f(x, s)$ 在 patch 之间递归传递（式 (1)）。注意力只在**当前 patch 片内**做局部混合，跨 patch 的历史一律由状态携带——这就是「有状态」的准确含义：历史信息的唯一通道是状态。

图 1 给出整体结构与原语的对应关系。正式配置为 `GRIDMAN_MEDIUM`：`embed_dim`=1024、`blocks`=3（对应 4 个 `c_state`）、`patch_size`=64，模型参数 437.8M[1]。

实现版本声明（重要）。本文所有实验数字来自 `v_0d3` 检查点；模型行为的描述以 `v_0d4` 实现（2026-10-18 更新）为准。两版差异集中在 `memory` 的作用域与训练配置，见表 1。差异不影响本文的推理侧结论：所有评测都在 batch = 1 上进行，v_0d4 的序列级记忆在 batch = 1 时读写单个槽位，与 v_0d3 的 `[1,…]` 布局逐位对应（§3.4）。

表 1：实现版本差异（v_0d3 → v_0d4）。本文检查点由 v_0d3 训练产生；行为描述以 v_0d4 为准。

项	v_0d3（本文检查点）	v_0d4（现行实现）
`memory` 作用域	跨样本共享（形状 [1, . . .]）	**序列级私有槽位**（[cap, . . .]，cap = max_batch）
提交规则	跨 batch 求均值	**无跨流平均**——序列间零泄漏
跨卡同步	每 patch `all_reduce` 均值	**已删除**，记忆 rank 私有不通信
写入门控偏置初始化	−6.0	$-\ln 63 \approx -4.14$（对齐 4096-token 队列视界）
MEDIUM `chunk_size` / `bptt`	192 / 4	1 / 16
存档格式	`global_memory` 顶层字段	memory 移入 per-rank runtime_states，**与 v_0d3 存档不兼容**

3.2 四件套状态及其作用域

状态由四个分量构成，按时间尺度与作用域分工，见表 2 与图 2。

作用域（按版本）。v_0d4 中**四个分量全部逐样本私有**——`memory` 为每条序列独占的槽位（[cap, . . .]，读取时按 [: batch] 切片），训练提交不做跨流平均、跨卡不做同步（序列间零泄漏）。本文检查点所属的 v_0d3 则是「`memory` 一个共享、其余三个私有」的两族划分：`memory` 写入后按流求均值、每 patch 一次跨卡 `all_reduce`。这个划分决定了「重置某个分量」实验（§5.5）的语义

[1]配置与参数量引自团队内部文档《论文大纲-B》第 0 章与配图；代码事实对应实现仓 `zhihumomo/ouro-research`。

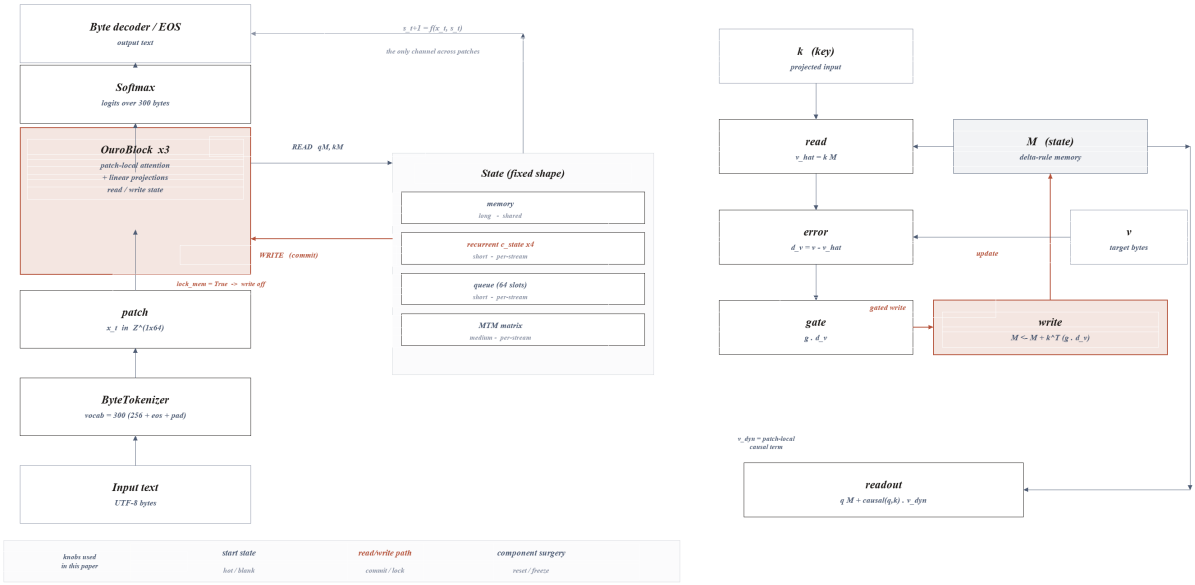


图 1：持续状态架构与状态读写接口。左：一条跨 patch 的递归流——输入文本经 byte 分词后在 patch 内做局部注意力混合，patch 之间的唯一信息通道是状态四件套；右：原语 OuroCell 的读写链条（读出 $\hat{v} = kM \to$ 误差 $\Delta v = v - \hat{v} \to$ 门控写入 $M \leftarrow M + k^\top(g \cdot \Delta v) \to$ 读出 $qM +$ 因果分数 $\cdot\, v_{\text{dyn}}$）。下方给出本文实验用到的三类操纵旋钮：起点状态（热启动 / 空白）、读写路径（commit / *lock*）、组件手术（重置 / 冻结）。本图由 Visio 绘制（源文件 `figs/fig_arch.vsdx`，用 VisioRender 渲染为 PNG；等价的矢量版本为 `figs/fig_arch_matplotlib.pdf`）。

——在 v_0d3 检查点上，重置 `memory` 相当于动跨样本的共享知识（本研究中等价于换回训练稳态快照），重置其余三个只影响当前样本的上下文记忆；若换用 v_0d4 检查点重跑，重置 `memory` 也将退化为只影响当前样本的操作，§5.5 的归因结论需要重新验证。**本文所有实验都在** batch = 1 **上进行**，此时两版的 memory 读写逐位对应，故推理侧结论不受该差异影响（§3.4）。**「多时间尺度」的分工叙述由实验支撑**，而不是凭结构空口断言：本文在 §5.5 用组件级重置与冻结给出分工的实测结果。

原语。四个分量是同一条 delta 规则在不同尺度上的实例（「一个原语、三个尺度」）：读出 $\hat{v} = kM$，误差 $\Delta v = v - \hat{v}$，门控写入

$$M \leftarrow M + k^\top(g \cdot \Delta v), \qquad g = \sigma(\cdot)/\text{eff_seq_len}, \tag{3}$$

随后读出 $qM +$ 因果分数 $\cdot\, v_{\text{dyn}}$。式 (3) 是线性预测器的在线（岭）回归更新在秩空间中的实例，1/eff_seq_len 缩放保证 patch 长度变化时写入量一致，门控 g 使写入可以自适应减弱。

3.3 状态的读写接口

两条前向路径：`commit`（写入，*lock_mem* = false，状态正常更新）与 `lock`（只读，*lock_mem* = true，状态原样直通）。**本文所有实验的操纵旋钮都落在这两个接口上**——这是 §2.3 中「状态是微观实现、可以显式调整」的具体兑现。

关键实现事实（受控设计的基础）：*lock_mem* = true 时四个状态分量原样直通，因此**第一个 patch 的输出与正常前向逐位一致**——此时还没有发生任何写入；从第二个 patch 起两者才因写入与否而分化，A2 测的就是这个分化量（§5.2）。

表 2: 状态四件套：机制、时间尺度与作用域。写入规则均为前向局部规则（无反向传播）。

分量	机制	时间尺度	作用域	写入规则
`memory`	delta 规则快权重矩阵（每层一块）	长	**序列级私有**（v_0d4；v_0d3 为跨样本共享、按流平均）	误差驱动外积，恒等初始化
`recurrent`	块级循环状态 `c_state`（3+1 个）	短	逐样本私有	LSTM 式门控 cell
`queue`	64 槽滚动队列	短	逐样本私有	FIFO，存顶层 `c_state` 历史
`MTM`	中期矩阵记忆	中	逐样本私有	delta 规则，写入源为队列滑出的旧 `c_state`

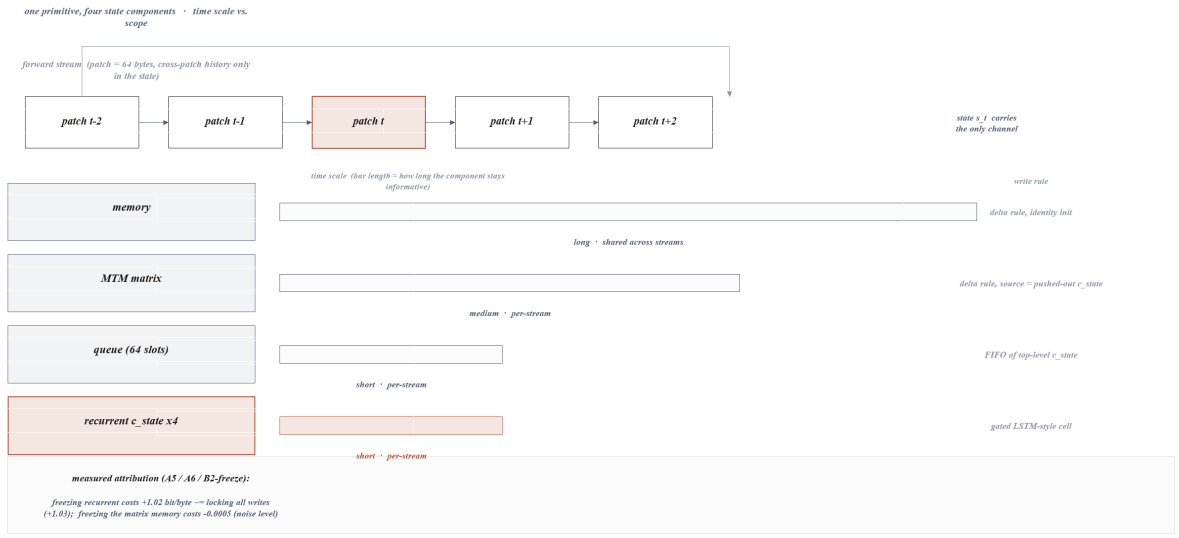


图 2: 四个状态分量的时间尺度与作用域（Visio 绘制，源文件 `figs/fig_state_scales.vsdx`）。横条长度示意各分量「信息保持多久」的相对量级，右侧一列是写入规则；底部给出本文实测的归属结论（A5/A6/B2-freeze 三组独立实验一致）。注意「参数量最大的分量」并不等于「承担跨 patch 适应的分量」——这一区分由第 5 章给出。

推论。状态因而是显式对象：前向返回新的状态对象而不原地修改，所以它可以被整体读出、冻结、置换，也可以在同一份快照上并行地开出多个实验分支（本文所有组件手术都是这个语义，见 §4.5）。

3.4 训练与检查点

状态不清零、跨批次连续（与 RNN/RWKV 的关键差别）。标准循环架构训练时状态从零初始化、一批读完即清零——单个状态最多跨一个样本的长度，记忆跨不过样本边界。本文的模型**不按批次清空状态**：上一批结束时的状态直接续进下一批，整条训练流是一条全局连续的流，状态的跨距不受样本长度限制。batch 维被实现为若干条并行的永续数据流，`recurrent`/`queue`/`MTM` 逐流私有；`memory` 在 v_0d4 中同样逐流私有（每条流独占一个槽位，提交无跨流平均、跨卡无同步），而本文检查点所属的 v_0d3 为跨流共享（写入后按流平均，每 patch 一次跨卡 `all_reduce`），`chunk_size`=192 条流。流数与 `bptt` 按版本配置：v_0d3 为 192/4，v_0d4 的 MEDIUM 档改为 1/16。这正是「固定状态持续携带跨上下文信息」（S1）在训练侧成立的前提。

记忆精度与门控初值（v_0d4）。记忆全程以 fp32 驻留（配置项 `mem_dtype`，可切 bf16 换显存，代价是小量级更新在 $|\text{mem}|\sim 1$ 处被舍入吸收）；写入门控偏置初始化为 $-\ln 63 \approx -4.14$，即 $\sigma(b) = 1/64$，使写入时间常数对齐 $64 \times 64 = 4096$ token 的队列视界（v_0d3 为硬编码 -6.0）。

优化与截断。AdamW + cosine 调度 + 梯度裁剪 1.0，bf16 混合精度，梯度窗口 `bptt`=4 个 patch（$4 \times 64 = 256$ token；本文检查点的 v_0d3 配置，v_0d4 MEDIUM 档为 16 patch = 1024 token）。**截断窗口之外的长程信用只能经前向 delta 规则写入传递**——这是连到「持续学习」主张的关键设计点：模型学到的不是「如何回传梯度到远处」，而是「如何在前向里把远处写进状态」。

检查点与热启动。训练收敛后状态达到稳态，检查点里保存的状态快照就作为评测的**热启动**起点；若存档没有保存状态，则回退按空白处理。存档格式随版本变化：v_0d3 把共享 memory 存为顶层 `global_memory` 字段（rank0 一份），v_0d4 把序列级 memory 放进各 rank 的 `runtime_states`（rank 私有、全量落盘，恢复时缺 memory 直接报错）——**两版存档互不兼容**。本文使用 step 144,000/705,600（约 20% 训练进度）的中型检查点 `gridman_medium_v_0d3_pretrain`，取 batch= 1 切片的状态，每次实验先克隆再跑——**状态跑一次就变**，不可复用同一对象，否则实验之间互相污染。本文所有数字都建立在这个单一检查点上，因此**只做条件间比较**，绝对数值不可与外部模型对照（§8）。

4 评测协议

本节定义贯穿全文的评测协议。所有实验脚本共享同一套脚手架：模型单例 + 热启动状态 + teacher-forced 流式评测 + 文本源切分 + 组件手术工具。

4.1 数据与 held-out 切分

语料。预训练语料为 `pretrain.jsonl`（约 9.5 GB，每行 `{"text": ...}`），微调语料为 `sft.jsonl`（约 14 GB，每行多轮对话）。训练按流（stream）组织：本文检查点（v_0d3）为 `chunk_size`=192 条并行流，每条流对应文件的一段连续 chunk（约 49.4 MB）。

零污染 held-out。训练在 step 144,000 时，每条流已消费 $144{,}000 \times 64\ \mathrm{B} = 9.2\ \mathrm{MB}$（这是从训练步数与 patch 长度推算的上界）。因此本文从每条流 chunk 内**偏移** ≥ 20 **MB 处**取行，再按 seed 洗牌拼成评测文本——取行位置远超已消费区，属同分布零污染。**跨域档**取 `sft.jsonl` 尾部（深度按 seed 随机化）；**消费区档**（成员推断用）取流 0 chunk 内 $[0.5, 7]$ MB 的随机偏移，即训练确实见过的文本。n-gram 重叠率报告的精确统计 [待补：待补：4-gram 重叠率明细见附录 C]。

合成档。可打印 ASCII（字节 $\in [33, 122]$，刻意 OOD）与词表内高频词对（分布内）两类，用于前向学习与遗忘类实验（附录 D.1、E）。

4.2 主指标：teacher-forced bits/byte

前向协议完全镜像训练前向：patch 大小 64、相邻窗口重叠 1 字节（输入 `ids[64k:64k+64]`，目标 `ids[64k+1:64k+65]`），model.eval() + bf16 autocast + `inference_mode`，不做生成式采样。指标为 teacher-forced 交叉熵，以 **bits/byte**（= nats/ $\ln 2$）报告；机会线为 $\log_2 300 \approx 8.23$ bit/byte（对 300 个字节均匀瞎猜）。

状态流必须逐 patch 推进。评测不能跳过任何 patch 的前向——跳过会冻结状态推进，测出来的就不是同一条流（本文实验中都只跳过「计分」，不跳过「前向」）。

口径说明（两处容易混淆的细节）。（i）当所有计分 patch 长度相同时，「每 patch 均值的均值」与「每 token 均值」严格相等，因此 bits/byte 与逐 token 均值可以直接互换；本文实验脚本中两种实现都出现过，所有报告数字都在等长 patch 窗口上取得，故不受该差异影响。（ii）n-gram 复现类指标（§6.2）的「距离」以字节计，在本模型中字节即 token。

4.3 起点状态与实验条件声明

按「每次评测显式声明 {起点状态, *lock* 与否, 重置点}」的约定，本文所有结果表都标注这三个条件。起点状态三种来源：**热启动**（训练稳态快照）、**空白**（恒等 memory + 零 recurrent/queue/MTM）、**热启动 + 插件**（如仅回滚 `memory` 或仅清零 `recurrent`，见 §4.5）。

关于空白态的适用性。空白起步使 delta 规则写读链路偏离训练操作点，它在损失上表现为结构性失效（§5.1），因此本文**不以空白作为主表基线**，而以热启动为基线、以「热启动 − 空白」的差作为「状态贡献」的度量，并把空白的绝对水平仅用作参照。出于同一原因，凡涉及「模型记住了训练语料吗」的问题（§7.3.1），必须**双报**两种起点。

表 3: 状态手术工具及其语义。所有操作都只改状态、不改权重。

操作	语义	用于
`lock_mem`	跨 patch 的持久写入关闭（patch 内 v_{dyn}/因果分数仍在工作）	A2、B2、G2
`reload_hot_memory`	仅 `memory` 回训练稳态快照	A5
`zero_recurrent`	仅 `recurrent` 清零	A5、C 线
`blank_queue_mtm`	`queue` 清零 + `MTM` 回恒等	A5、C 线
freeze$_{\text{子集}}$	每个 patch 前向后把指定分量子集恢复为前值（该分量跨 patch 不更新，patch 内照常更新）	A6、B2-freeze

4.4 统计口径

（i）文本真随机化。同一实验的多个 seed 必须改变**取文本的偏移**，否则多个「seed」实际是同一段文本，会给出假窄的置信区间（本文脚手架在早期版本犯过该错并已修正）。**（ii）成对报告。**凡是同一文本上的两条件对照（如 write vs *lock*），先算每个 seed 的差值再取均值，而非分别取均值后相减——前者剔除了文本难度差异。**（iii）置信区间。**报告 $1.96 \times \text{std}/\sqrt{n}$ 的 95% 区间；$n \leq 2$ 时区间仅作参考。**（iv）显存卫生。**连续执行多个短评测任务时逐次清空 CUDA 缓存分配器（否则会累积到 OOM，实测 25 GB）。**（v）确认性与探索性的区分。**第 5 章的状态消融（A1–A6）与 B2 的主结论属于**确认性**实验：成立条件在跑之前就写明（§1.2），实现缺陷订正后重跑仍成立。而**自然重复的距离分桶**（§6.2 的表 8）是**探索性/事后分组**：桶边界由我们选定，不同距离上重复的是不同的 n-gram 群体，因此该表用于描述形状而非做显著性检验。附录 E–E 的 D/E/G 三线（状态知识性质、稳定性、前向学习）是在主结论之后追加的探索，用于解释机制，不承担主 claim。

（vi）数字溯源。本文每张表的每个数字都可追到实验包里的一个结果文件：附录表 11 给出「脚本 ↔ 实验」的对应关系，结果文件命名统一为 `stage*_results.json`（A1 为 `a1_*_per_patch.json`）；改口径即重跑、重跑即换文件，不做手工誊抄。表中未取得的数据一律标 [待补：]，不以估计值填充。

4.5 状态手术：本文的操纵旋钮

由于状态是显式、不可变的对象（§3.3），本文可以在任意 patch 边界对状态做手术。协议共用到五类操作（图 3），全部实现为「在某个 patch 前/后把状态替换为新对象」：

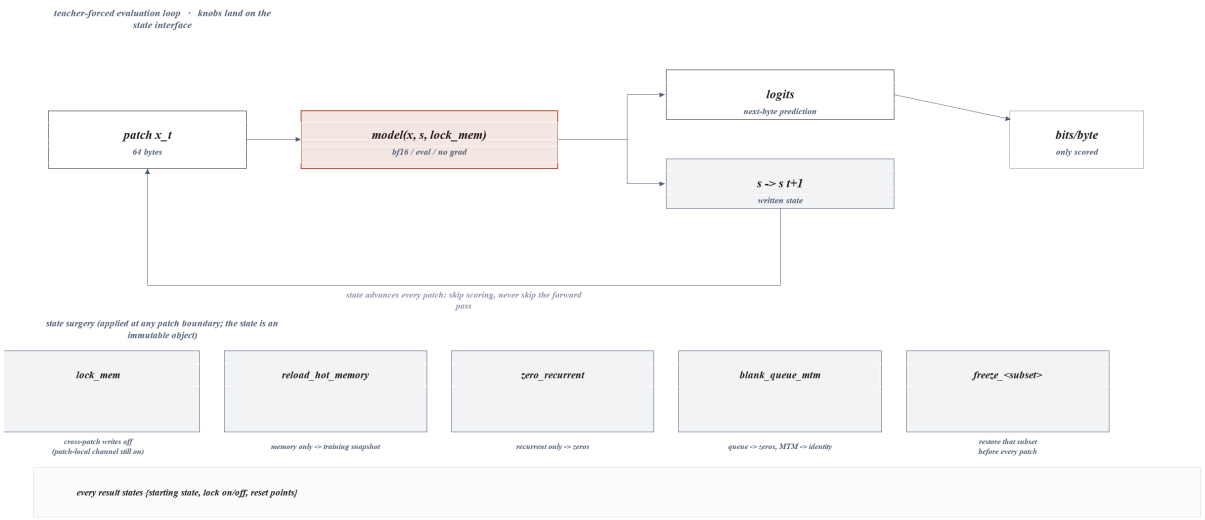


图 3: teacher-forced 评测回路与状态手术（Visio 绘制，源文件 `figs/fig_protocol.vsdx`）。每个 patch 都必须前向（跳过会冻结状态推进，只允许跳过计分）；五类手术都在 patch 边界上把状态换成新对象，因此互不干扰的实验分支可以在同一份快照上并行开出。

lock 与 **freeze 的区别必须写清。***lock_mem* 关闭的是**跨 patch 的持久写入**，patch 内的动态通道仍然工作；freeze 是逐分量版本——它冻结的是某一个分量在跨 patch 尺度上的更新，其余分量照常参与。两者互为「全局/组件」两种粒度的同一干预，这正是 §5.5 能把「写入收益」拆到组件上的原因。

勘误与协议纪律。本文在成稿前对实验包做了逐脚本审读，发现并订正了三处口径问题，其中两处影响数字、一处影响结论：n-gram 分桶的一处实现缺陷（导致「长程零衰减」的错误结论，已修复重跑，见 §6.2）；A1 的计分窗口未对齐（已改为所有条件共用同一窗口，见 §5.1）；受控遗忘曲线的答案位置整体偏移 66 字节（实验作废，说明见附录 D.1）。这三处订正都已落到脚本里，并配了一个模型无关的位置自检脚本作为回归测试。

5 实验一：起点状态（子主张 S1）

本章逐组给出起点状态的消融结果（总览见图 4），回答 S1 的两个成立条件：**热启动是否显著优于空白**（§5.1）与**写入是否显著优于** *lock*（§5.2），再用跨文档边界（§5.3）与组件级归因（§5.5）回答「信息由谁承载」。

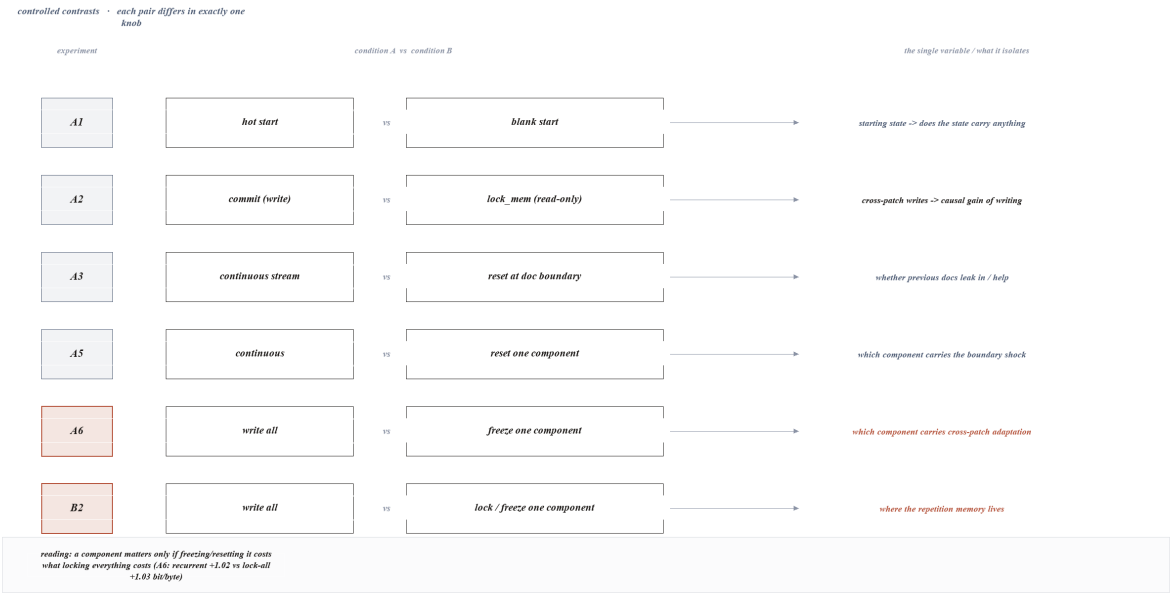


图 4：受控对照一览（Visio 绘制，源文件 `figs/fig_ablation_matrix.vsdx`）。每一对条件只差一个旋钮；右侧一列写出该对照隔离出的变量。阅读方式写在图底：一个分量只有在「冻结/重置它的代价等于冻结全部写入的代价」时，才算真正承载了这门功能。

5.1 热启动 vs 空白起步（A1）

设计。同一份文本、同一前向协议，唯一变量是初始状态：热启动（`load_model_checkpoint` 取训练稳态快照）对空白（`init_inference_state`：恒等 memory + 零 recurrent/queue/MTM）。**计分窗口对齐：**三个「空白 + 预热 W token」档（$W \in \{0, 256, 1024\}$）在实现上只是「从第 $W/64$ 个 patch 起计分」——每个 patch 都会被前向，所以三档的状态轨迹完全相同。既然状态轨迹相同，三档就必须在**同一个计分窗口**上比较，否则「预热」只是个伪变量。本文按此对齐：所有条件一律从第 16 个 patch 起计分。

结果（表 4、图 5）。对齐窗口（48 个 patch = 3 KB，文本取自 sft 尾部 4097 字节）：热启动 1.588 bit/byte，空白 5.319 bit/byte，差距 **3.731** bit/byte。三个空白档在对齐窗口上的数值**逐位相同**（最大差 0.000e+0）——这直接印证了「预热档是伪变量」。窗口内**无追平趋势**（图 5 右：*blank* − *hot* 全程在 1.1–4.6 nat 之间波动，无下行）。

若不做窗口对齐（即照早期报告的写法），三档分别是 5.668/5.544/5.319 bit/byte，其中「0.35 bits 的挽回」全部来自把最差的头几个 patch 排除在计分之外，而不是模型写得慢——这一点必须在论文里写清，否则会误读成「空白态可以通过预热追上」。

判定。S1 的第一条成立：热启动显著优于空白，且差距不是「写入速度」能解释的（对齐窗口内空白没有变小趋势）。**推论与限定：**空白态的差是结构性的——恒等/零状态使 delta 规则的读写链路偏离训练操作点，前向通道在空白态下并未进入正常工作区。因此空白**不能**作为「无记

表 4: A1 热启动 vs 空白起步。文本为 `sft.jsonl` 尾部 4097 字节（跨域、零污染），所有条件记分窗口相同（第 16–63 个 patch，共 48 个 patch = 3 KB）。「预热」档在对齐窗口上数值逐位相同（最大差 0.000e+0），故不是有效变量；最后一行给出未对齐窗口时的数值，两者相差的 0.35 bit/byte 纯属窗口效应。

条件	起点状态	计分窗口	bits/byte
热启动（hot）	训练稳态快照	48 patch（对齐）	**1.588**
空白（blank）	恒等 memory + 零其余	48 patch（对齐）	5.319
空白 + 预热 256	同上（同一轨迹）	48 patch（对齐）	5.319
空白 + 预热 1024	同上（同一轨迹）	48 patch（对齐）	5.319
热启动 − 空白（对齐窗口）		48 patch	**+3.731**
热启动（未对齐）	训练稳态快照	64 patch	1.623
空白 + 预热 0（未对齐）	恒等 memory + 零其余	64 patch	5.668
空白 + 预热 256（未对齐）	同上	60 patch	5.544

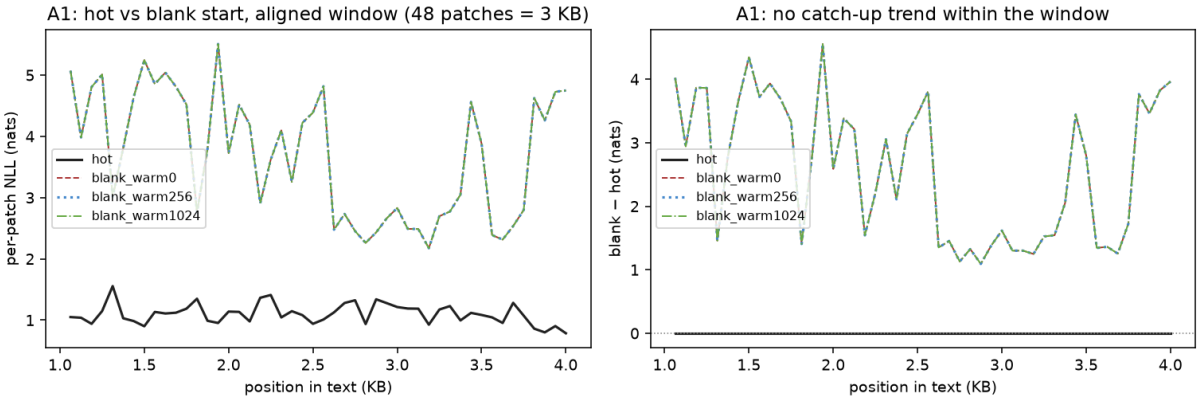


图 5: A1：热启动 vs 空白起步（同一计分窗口，48 个 patch = 3 KB）。左：逐 patch NLL；三条空白曲线**完全重合**（预热档同一计算），热启动全程低 1–4 nat。右：*blank* − *hot* 的差值曲线，窗口内无收敛趋势。

忆」的干净对照，只能作为「状态贡献」的上界演示；热启动记忆的成分分解（通用先验 / 同分布偏向 / 逐样本痕迹）见 §7.3.1。

5.2 写入的因果贡献：*lock* 对照（A2）

设计。同一份同分布 held-out（16 KB，seed $\in \{0,1,2\}$ 三份不同文本），两个条件：`write`（正常 commit）与 `lock`（*lock_mem* = true，跨 patch 持久写入关闭）。由于 *lock* 不改变首个 patch 的输出（§3.3），两条件在前 64 字节逐位一致，差异只来自「从第二个 patch 起，这段上下文是否被写进状态」。指标为 bits/byte，并按 seed 成对求差（§4.4）。

结果（表 5、图 6）。 write = 1.581 ± 0.493、lock = 2.580 ± 0.399 bit/byte，**成对差值** = +0.999 ± 0.095 bit/byte（三份文本分别为 0.978/1.091/0.927）。逐 patch 看，差值**随后段增长**：前 32 个 patch +0.585 nat、后 32 个 patch +0.771 nat——写入的收益是**累积**的。

判定。S1 的第二条成立：跨 patch 写入对预测有稳定、可累积的正贡献。**注意语义：***lock* 只冻结**跨 patch 的持久写入**，patch 内的动态通道（v_{dyn} 与因果分数）仍在工作（表 3），所以该对照隔离的是「跨 patch 写入」，不是「全部记忆效应」；把这一收益拆到具体分量，见 §5.5。

表 5： A2 写入 vs 只读（*lock_mem*）。同分布零污染 held-out 16 KB，三个 seed 为三份不同文本。主报**成对差值**（先算每份文本的差再平均），它剔除了文本难度差异；末两行为 seed 0 的逐 patch 差值演化，显示写入收益随上下文累积。

seed（文本）	write (bits/byte)	lock (bits/byte)	差值（lock − write）
0	1.661	2.639	+0.978
1	1.111	2.202	+1.091
2	1.971	2.899	+0.927
均值 ± 95% CI	1.581 ± 0.493	2.580 ± 0.399	$+\mathbf{0.999} \pm \mathbf{0.095}$
seed 0 逐 patch 差值（nats）：前 32 patch +0.585；后 32 patch +0.771			

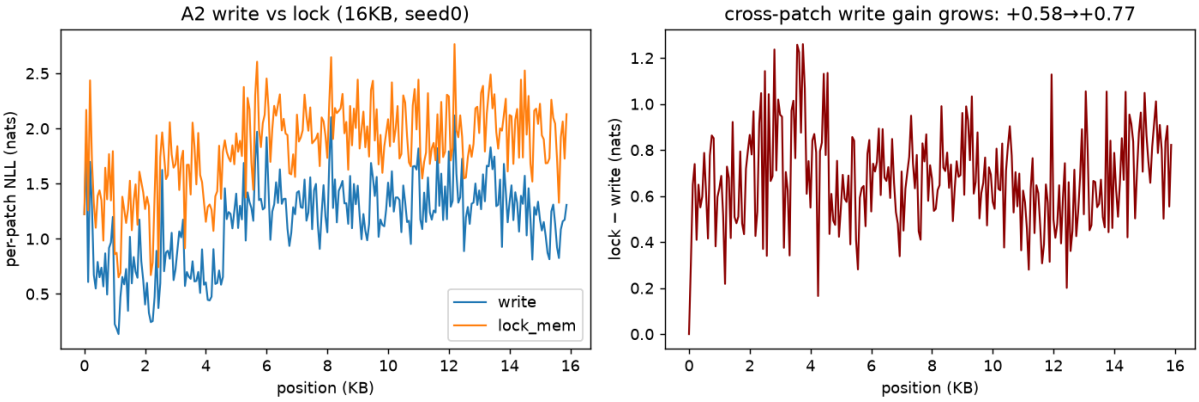


图 6： A2：写入 vs 只读（16 KB，seed 0）。左：逐 patch NLL；右：*lock* − write 的差值，随上下文增长而增大（前段 +0.58 → 后段 +0.77 nats），即跨 patch 写入的收益在累积。

5.3 跨文档边界的信息延续（A3）

设计。16 篇文档 × 3072 B（共 768 个 patch、15 个边界）拼成一条流，条件：**continuous**（状态连续跨文档，即训练时的行为）、**reset_full**（每个边界把状态重置回热启动全态）、以及重置单个分量的三个变体（在 §5.5 中报告）。指标：整体 bits/byte、边界后 2 个 patch 相对边界前基线的冲击（nats）、以及回落到「基线 +0.5 nat 以内」所需 patch 数（恢复长度）。

结果（表 6、图 7）。连续 1.1032 对全重置 1.1281 bit/byte；后 14 篇（跳过前 2 篇 burn-in）的平均 NLL：连续 0.7711 对全重置 0.7903 nat，跨文档增益 $-\mathbf{0.0192}$ nat。边界冲击 $+\mathbf{0.4504}$ nat，恢复长度**中位数 1 个 patch、最大 2 个 patch**（$n = 15$ 个边界）。

表 6： A3 跨文档边界（16 篇文档 × 3072 B = 768 patch、15 个边界）与 A5 单组件重置。冲击 = 边界后 2 个 patch 的平均 NLL 相对边界前 8 个 patch 基线的增量；恢复长度 = 回落到「基线 +0.5 nats」以内所需的 patch 数。连续条件无重置点，故不给冲击/恢复。

条件	bits/byte	后段 NLL (nats)	边界冲击 (nats)	恢复（中位 / 最大）
continuous（不重置）	1.1032	0.7711	—	—
reset_full（全部重置）	1.1281	0.7903	+0.4504	1/2
reset `recurrent`	1.1241	0.7875	+0.3779	1/2
reset `memory`	1.1029	0.7709	+0.0246	0/1
reset `queue+MTM`	1.1026	0.7708	+0.0250	0/1
跨文档增益（continuous − reset_full，后 14 篇）：−0.0192 nats； `recurrent` 解释边界冲击的 84%				

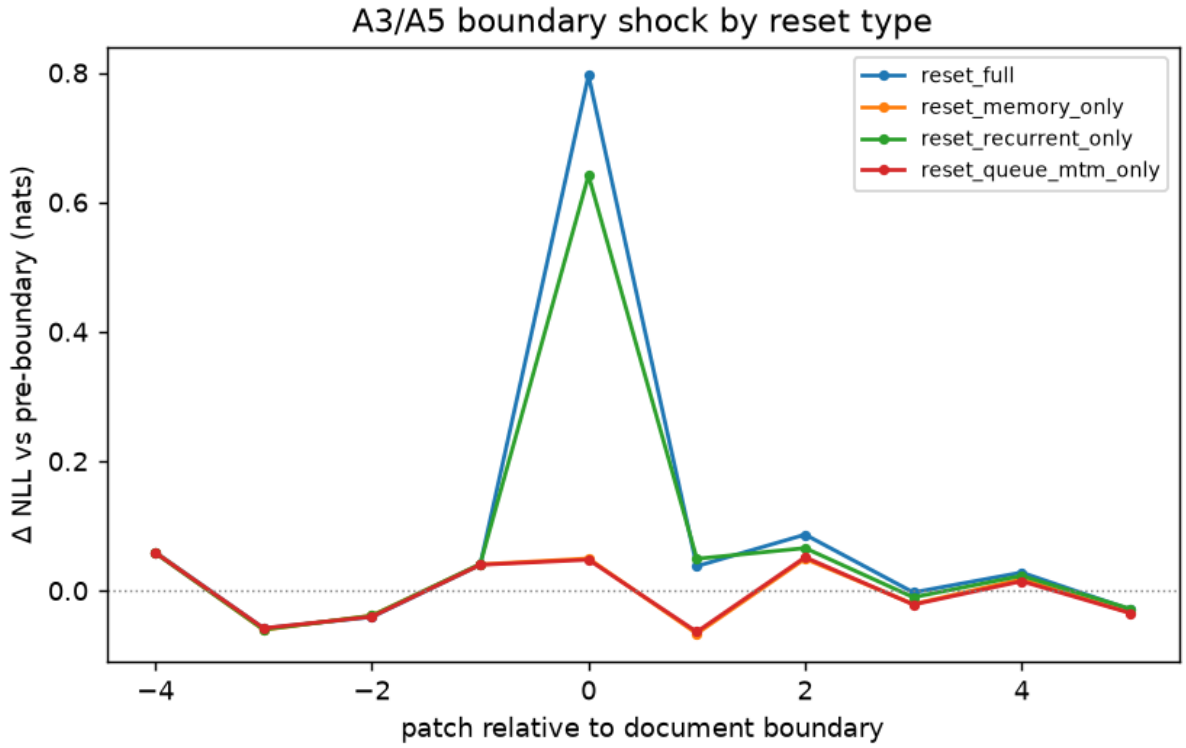


图 7： A3/A5：以文档边界为原点对齐的平均恢复曲线（16 篇文档、15 个边界）。全部重置的冲击明显；单组件重置的曲线给出归因——`recurrent` 的曲线几乎复现全重置冲击，`memory` 与 `queue+MTM` 的重置几乎免费。

判定。跨文档的信息延续**真实但量级小**（∼ 0.02 nat）；而边界冲击虽然尖锐（+0.45 nat，约合热启动整体水平的量级），却**在 1–2 个 patch 内自愈**。这两条合起来说明：固定状态确实把前一篇文章的某种统计结构带进了后一篇，但「跨文档迁移」不是本文这个检查点的主要收益来源。

5.4 状态的上下文特异性：错配实验（A4）

设计（未完成）：用上下文 A 跑出状态，直接在上下文 B 的文档上评测（状态—文本错配），对照空白态与匹配态；同主题与跨主题各一组，得到 3×2 的对照矩阵。**判读：**错配 $\leq$ 空白 $\Rightarrow$

表 7：组件级归因：两种互补干预在同一份同分布文本上的结果。（a）A6 冻结（16 KB，seed $\in \{0, 1\}$）：基线为全部正常写入；（b）B2-freeze（96 KB，单条流）在自然重复任务上的 $<$1k 桶复现增益。两组的结论一致：代价几乎全部落在 `recurrent` 上，矩阵记忆与队列 + MTM 无损失。

(a) A6 冻结单分量（16 KB，2 seed）		
条件	bits/byte（均值）	Δ vs 全部写入
write（全部写入，基线）	1.386	0
lock（冻结全部写入）	2.420	+1.034
freeze `recurrent`	2.408	**+1.022**
freeze `memory`	1.386	−0.0005
freeze `queue+MTM`	1.386	+0.0002
(b) B2-freeze：自然重复的 $<$1k 桶复现增益（nats，96 KB）		
条件	bits/byte	$<$1k 桶增益
write	1.0838	+0.0839
lock（全程只读）	2.1641	−0.0050
freeze `recurrent`	2.1527	**−0.0043**
freeze `memory`	1.0836	+0.0840
freeze `queue+MTM`	1.0832	+0.0837

状态是通用缓存；错配明显更差 ⇒ 状态有上下文特异性。[待补：该实验尚未运行：需要成对主题数据（同主题/跨主题各若干对），计划与 D 线（§7.3.1）合并成「状态里装的是什么」一节。] 本文目前只用 D 线间接回答该问题（§7.3.1）。

5.5 组件级归因（A5 / A6）

设计（两种互补的干预）。 **A5 重置：**在文档边界处只重置**一个**分量（`memory` 回训练稳态快照 / `recurrent` 清零 / `queue+MTM` 清零），其余保持连续；**A6 冻结：**在同一份 16 KB 同分布文本上，每个 patch 前向后把**一个**分量恢复为前值（该分量跨 patch 不再更新，patch 内照常更新），与「冻结全部写入」的 *lock* 及「全部正常写入」的 write 对照。两者一个问「重置它损失多少」，一个问「冻结它损失多少」。

结果（表 7）。A5：边界冲击 +0.3779 nat（只看 `recurrent`）对 +0.4504 nat（全重置），即 **84% 的边界冲击由 `recurrent` 清零解释**；只重置 `memory` 或 `queue+MTM` 的冲击都是 $\approx$ +0.025 nat（近乎免费）。A6：以 write 为基线，*lock* 全部写入 +1.034 bit/byte，**冻结 `recurrent`** +1.022 bit/byte（$\approx$ 全锁的 98.8%，两个 seed 分别 +0.960/ + 1.084），冻结 `memory` −0.0005 bit/byte，冻结 `queue+MTM` +0.0002 bit/byte——都在噪声水平。

判定（本文最强的一条归因结论）。自然文本上的跨 patch 适应**几乎全部由 `recurrent`（块级循环状态 `c_state`）承担**——A2（全锁 +1.00 bit/byte）、A6（冻结 `recurrent` +1.02 bit/byte $\approx$ 全锁 +1.03 bit/byte）、A5（边界冲击 +0.38/0.45 nat）三个独立实验交叉一致，4 个数量级的不同干预给出同一个答案。**这修正了「memory 管长期」的朴素假设：**矩阵记忆在 3 KB–16 KB 尺度上是「一个 patch 即可重建」的快权重，它的价值不在增量适应——冻结其写入几乎零代价，但它**读取训练先验**的贡献巨大（去掉热启动后损失升高 3.7 bit/byte 以上，§5.1）。B2-freeze（§6.2）

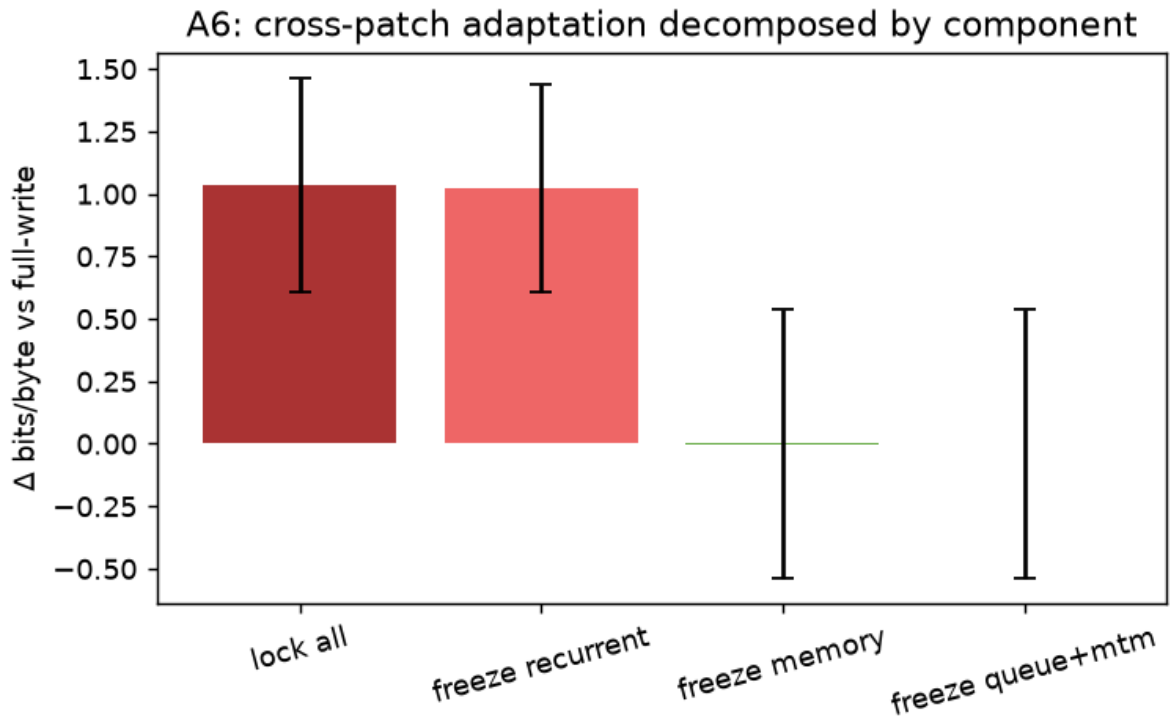


图 8: A6：跨 patch 适应按组件分解（16 KB，2 个 seed，误差棒为 95% CI）。冻结 `recurrent` 的代价与冻结全部写入几乎相同；冻结矩阵记忆与队列 + MTM 的代价为零。

在长程复现任务上给出同一归属结论。版本注：以上归因在 v_0d3 检查点上测得，其时 `memory` 跨流共享；v_0d4 将其改为序列级私有槽位（§3），「重置 `memory` = 换共享快照」的干预语义随之变为「换单流快照」，换用 v_0d4 检查点后该结论需重新验证。

5.6 小结：S1 成立

S1 成立：热启动显著优于空白（3.73 bit/byte，同窗口、无收敛趋势），正常写入显著优于 *lock*（$+1.00 \pm 0.09$ bit/byte，且随长度累积），两条成立条件同时满足。归因层面：跨 patch 的信息传递与适应主要由 `recurrent` 承载，`memory` 的作用是**读取训练稳态先验**，`queue+MTM` 在本文的尺度上未显示出可测贡献。

6 实验二：有效上下文窗口（子主张 S2）

本章用三把尺测量固定状态在损失上等效的上下文长度：位置衰减（§6.1）、重复距离（§6.2）、等效滑窗（§6.3），外加精确检索的边界预实验（§6.4）。截至本稿，前两把尺已完成（其中位置衰减以单流曲线替代），后两者未完成，如实标注。

6.1 长文档位置衰减（B1）

做成了什么。完整的 B1（5–10 篇 3 万–10 万字节文档的整篇曲线）尚未运行；作为替代，本文已有两条**单流逐 patch 曲线**：96 KB（seed 7）与 512 KB（seed 11），并在同一条流上做了重置频率扫描（E5）。

结果（图 9 右、图 10）。 96 KB 流上逐 patch NLL 从前段 0.4970、中段 0.6238 漂移到后段 0.7784 nat；512 KB 流同形状（0.695 → 0.874 nat），全程无尖峰。重置频率扫描（64 KB 文本，状态每 N token 重置回热启动快照，$N \in \{512, 4096, 16384, \infty\}$）给出 1.2147/1.0611/1.0430/1.0385 bit/byte ——**单调下降、无 U 形。**

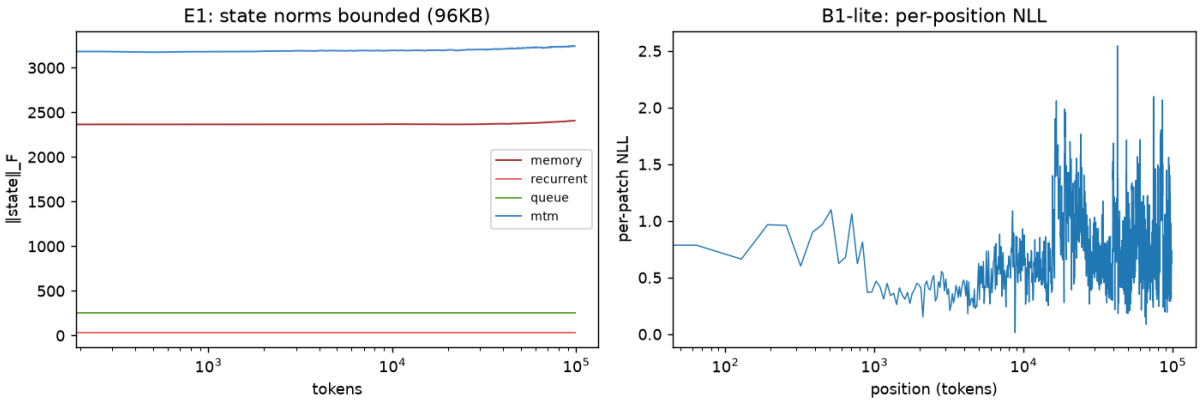


图 9: E1：96 KB 单流的状态范数与逐 patch NLL。左：四分量范数（对数横轴）全程有界；右：per-position 曲线（B1-lite），后段抬升与文本难度漂移一致（由 E5 判定，见正文）。

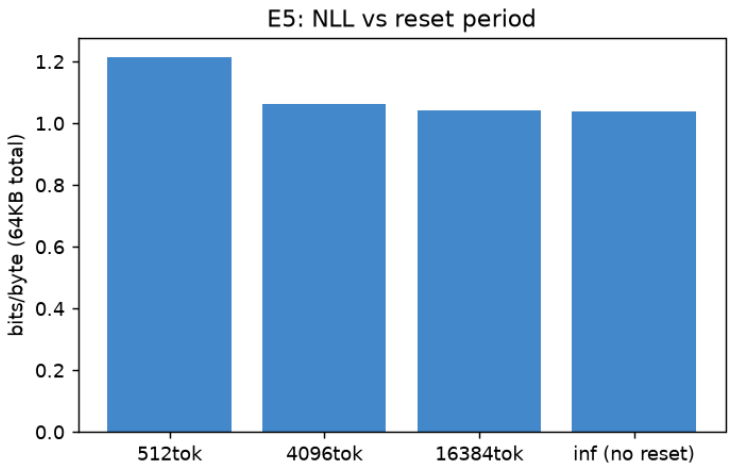


图 10: E5：64 KB 文本上，状态每 N token 重置一次的总 bits/byte。单调下降、无最优重置点 ⇒ 在这个尺度上「永续写入」是纯收益、没有噪声累积。

表 8： B2 自然重复的复现距离分桶（96 KB 流，$n = 8$；已按修复后的分桶实现重跑）。「距离」为与首次出现之间的字节距离（本模型中字节即 token）；增益 = 首次 − 二次，正值表示第二次出现被更好地预测。样本数互不相同这一点是分桶正确性的必要条件（见正文勘误说明）。bucket 间比较并非完全受控——不同距离上重复的是不同的 n-gram 群体，其首次出现难度本身不同。

距离桶	<1k	1k–4k	4k–16k	16k–64k	>64k
配对数量（$n = 8$）	9384	1225	1309	2708	382
首次出现 NLL (nats)	0.6863	0.5167	0.4773	0.4665	0.5082
二次出现 NLL (nats)	0.6024	0.4977	0.4900	0.4964	0.5229
增益 (nats)	**+0.0839**	+0.0191	−0.0127	−0.0299	−0.0148

$n = 16$（配对数量 3712 / 244 / 38 / 243 / 21）：增益 +0.0797 / +0.0054 / −0.0467 / −0.0042 / +0.0021

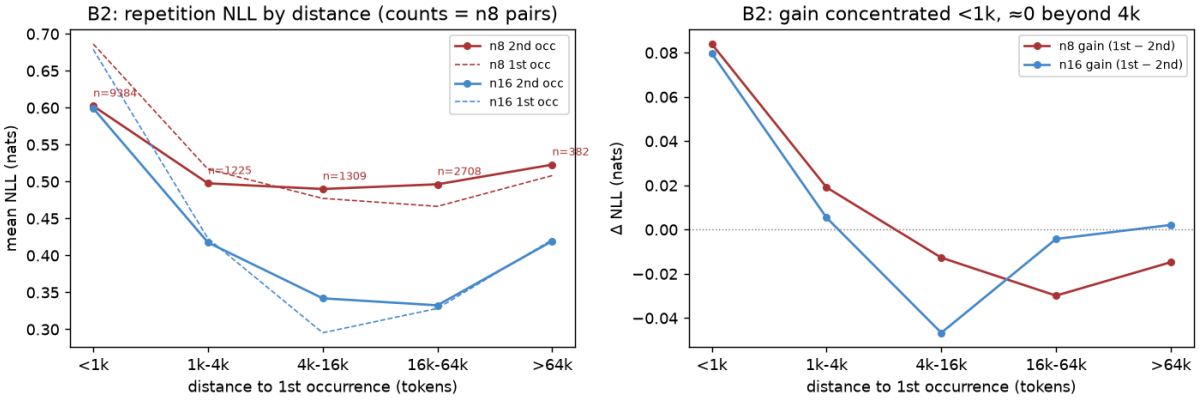


图 11： B2：自然重复的复现 NLL 与距离（96 KB 流）。左：两桶的首次/二次出现 NLL（标注 $n = 8$ 的配对数量）；右：增益 = 首次 − 二次。增益集中在 <1k，4k 之外衰减到零甚至转负。

判定。后段 NLL 的抬升**不是状态退化**：若存在噪声累积，重置频率扫描应出现 U 形（重置太少则累积过噪声、太多则丢信息），而实测是单调的——每多写一点就多赚一点。因此该抬升更保守的解释是流内文本难度/分布的漂移。这一点必须在论文里写清，因为它决定了「永续写入」是否安全。**[待补：B1 的整篇 vs 切段对照（真曲线）待补：需要 5–10 篇长文档，实现见《论文大纲-B》第 3 章第 3.1 节。]**

6.2 自然重复的携带距离（B2）

设计（免合成）。在同一份 held-out 上流式取逐字节 NLL，扫描流内 n-gram（$n = 8, 16$）的第二次出现，按与第一次出现的距离分桶（<1k / 1k–4k / 4k–16k / 16k–64k / >64k 字节），每个 gram 只计首次复现，指标为两次出现处 n 个字节 NLL 的均值（nats），增益 = 首次 − 二次。

结果（表 8、图 11）。96 KB 流、$n = 8$：增益**集中在 <1k 桶**（+0.0839 nat，**9384 对**），1k–4k 桶降至 +0.019，4k 以外为 −0.013/ − 0.030/ − 0.015；$n = 16$ 同形状（<1k 桶 +0.0797），但远桶样本数只有 38/21，噪声大。512 KB 流上 <1k 桶增益 +0.0668 nat（49716 对），1k–4k 桶为 −0.005，64k–256k 与 >256k 桶分别在 +0.006/ − 0.013 附近波动（图 13 右）。

组件归属（B2-freeze）。把同一份 96 KB 文本分别跑 5 个条件（全写、全程只读、冻结单分量），<1k 桶增益为：write +0.0839、lock −0.0050、freeze `memory` +0.0840、**freeze `recurrent`**

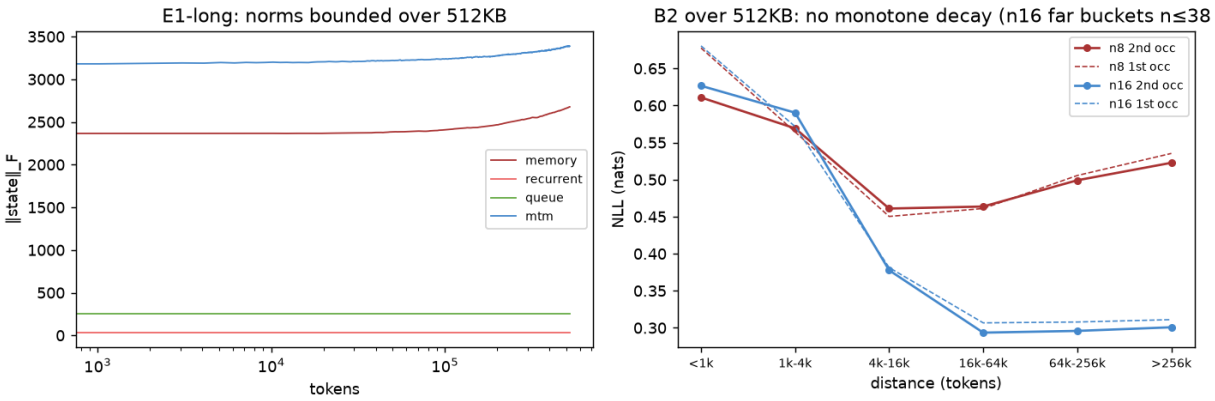


图 13: E1-long：512 KB 单流。左：状态范数（memory ×1.13、MTM ×1.07，STM/queue 恒定）；右：B2 距离分桶的首次/二次出现 NLL——无单调衰减形状，$n = 16$ 的远桶样本数 ≤ 1294，读图需谨慎。

−0.0043、freeze queue+MTM +0.0837 （表 7(b)、图 12）。**冻结 `recurrent` 精确抹平了复现增益，**代价层面也与全程只读等价（bits/byte 2.153 对 2.164）。这与 §5.5 的 A6 结论自洽：该记忆存在 `recurrent` 里且是写入依赖的。

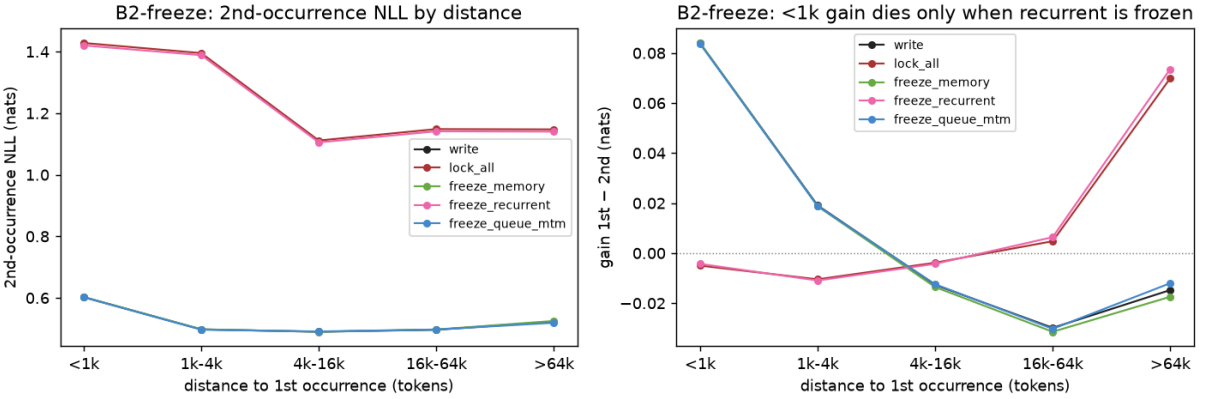


图 12: B2-freeze：复现记忆的组件归属。左：二次出现的 NLL 曲线（5 个条件）；右：增益 = 首次 − 二次。只有 `recurrent` 被冻结（或全程只读）时 <1k 增益消失，冻结矩阵记忆与队列 + MTM 无损。

一处必须公开的勘误。早期版本曾得到「复现增益在 <1k 到 >256K 的所有距离桶内零衰减」的结论，并据此写了「固定状态的记忆 ≥ 256K token 不衰减」。该结论**作废**：分桶统计的实现把各桶的样本数写成了同一个值（每桶的数值被复制成同一份），即「零衰减」是**所有桶都在报告同一个桶**造成的假象。修复后重跑（96 KB 与 512 KB 两条流）即得到上表结果。本文所有 B2 数字均为修复后数值；受影响的三张图已重画。**教训：**分桶类统计必须在脚本里断言「各桶样本数互不相同且总和等于配对总数」，否则一个变量作用域缺陷就会产出看起来干净、实则自证的曲线。

判定。S2 的量化结论是：固定状态的**有效重复记忆距离约为 1–4k token** （<1k 内增益显著，4k 外基本消失），且这一记忆**完全由 `recurrent` 承载**。一个自洽的旁证：模型内部的通量结构里，队列的时标是 64 槽 × 64 token = 4096 token （表 2），与实测的 1–4k 量级吻合。[待补：该「时标吻合」是否有因果含义，需要 C 线（受控注入 + 组件分离的遗忘曲线）才能判定，见附录 D.1。]

6.3 等效滑窗基线（B3）

设计（未完成）：训一个 2–4 层、局部因果窗口为 W 的 byte Transformer 基线（$W \in \{512, 2\text{k}, 8\text{k}, 32\text{k}\}$，数据同分布），在 B1 的同一批长文档上跑同一套 teacher-forced 循环，得到各档 W 的逐位置曲线，对每个位置插值出与固定状态曲线最接近的 W，即「固定状态 $\approx W$ token 滑窗」的等效窗口。来不及训练时的降级方案是引用公开滑窗基线并注明口径差异。[待补：本项需要训练基线，超出「只用现成权重」的范围；这也是本文唯一能给出 S2 绝对数字的途径。]

6.4 精确检索的边界（B4）

设计（未完成）：在距离 $d \in \{1\text{k}, 4\text{k}, 16\text{k}\}$ 处插入 `key: value`，文末 probe，统计精确匹配率。**先行证据（指向「不擅长」）**：合成任务上的离散绑定测试（附录 E）显示该检查点对随机 ASCII 键值对的绑定很弱——k 次演示后答案位 NLL 仍在 8.3–8.6 bits（机会线 8.23），k 增大不累积；换成分布内的词对后曲线完全平（5.0–5.2 bits，与首字节机会线重合），即这不是合成数据 OOD 造成的。据此本文**预判 needle 类精确检索不是本架构的强项**，将其降级为限定 claim 与未来工作（§8），并如实报告而不是回避。

6.5 小结：S2 部分成立

S2 的部分结论：有效窗口**显著大于 0**（$<$ 1k token 内复现增益 $+0.084$ nat、且 1k–4k 仍有残余），但**远小于**早期误导性结论给出的 $\geq$ 256K：固定状态在这个检查点上的自然重复记忆距离是 1–4k token。因此「永续上下文」这一强主张**在当前证据下不成立**，可辩护的表述是「固定大小状态等效于一段数 k token 的滑动记忆，且其容量不随历史增长而衰减」。稳定性侧的正面证据（512 KB 连续写入范数有界、重置频率扫描单调）说明**长度**不是问题，瓶颈在**容量与写入/读出机制**——这正是下一步训练侧实验（F 线）要回答的问题（§7）。

7 讨论

7.1 两个子主张的回答汇总

表 9 把 S1 / S2 与本文全部证据对上。一句话总结：**固定大小的持续状态确实承载了可测的跨上下文信息（S1 成立），但它的有效记忆距离只有 1–4k token（S2 部分成立）**；而这两件事的承载者都是块级循环状态，不是那个参数量最大的矩阵记忆。

表 9: S1 / S2 的判定汇总。每个判定句都由对应小节的末句直接给出，可逐条回查。

子主张	证据（实验）	关键数字	判定
S1（有没有）	A1 热启动 vs 空白	同窗口 1.588 vs 5.319 bit/byte（差 +3.731），窗口内无收敛趋势	**成立**
S1（有没有）	A2 写入 vs *lock*	成对差 +0.999 ± 0.095 bit/byte；差值前段 0.585 → 后段 0.771 nat	**成立**
S1（谁承载）	A5 单组件重置 / A6 冻结 / B2-freeze	冻结 `recurrent` +1.022 bit/byte（≈ 全锁 +1.034）；冻结 `memory` −0.0005；边界冲击 0.378/0.450 nat	跨 patch 适应由 `recurrent` 承载
S2（有多少）	B1-lite + E5 重置频率	96/512 KB 漂移无尖峰；重置周期 512 → ∞：1.2147 → 1.0385 bit/byte 单调	长度非瓶颈
S2（有多少）	B2 距离分桶 + B2-freeze	<1k 增益 +0.0839 nat（9384 对），4k 外 ≈ 0；冻结 `recurrent` 抹平增益	**有效记忆 ≈ 1–4k token**
S2（折算窗口）	B3 等效滑窗基线	[待补：未完成]	待定
边界能力	B4 needle 预实验 / G1 合成绑定	合成键值对绑定弱（8.3–8.6 bits ≈ 机会线）	限定 claim（§8）
评测去污染	D2b 成员推断	状态级样本记忆 −0.714 bit/byte;热启动 train gap −0.350 ± 0.200，空白 +0.365 ± 0.828	**主表须双报起点状态**

7.2 设计启示：状态接口的可操纵性

本文所有实验的旋钮都落在架构暴露的接口上，没有一处需要改权重、改数据或改提示词。这一点的工程含义值得单独记下来：

- *lock* **天然给出受控对照**：因为首个 patch 的输出与正常前向逐位一致（此时尚未发生写入），两组条件在第一步严格相同，差异只能来自后续的写入行为——这比「同一模型跑两次但初始化不同」之类的对照干净得多。
- **状态不可变 ⇒ 实验可并行、可回滚**：每个 patch 后拿到的是新状态对象，因此可以在同一条快照上同时开出「只重置 `memory`」「只清零 `recurrent`」「冻结 `queue+MTM`」等多个分支而不互相污染。

表 10: D 线：热启动状态里装的是什么（4 KB × 5 seed，95% CI）。（a）D1 领域分解：gain = 空白 − 热启动，四档文本；（b）D2/D2b 成员推断：训练消费区 vs 未消费同分布文本，以及「去掉热启动」后的同一比较。热启动的收益主要是通用序列先验（跨域仍在），含少量同分布偏向；而**逐样本的训练痕迹几乎全部由状态承载**（去掉热启动后 train gap 消失）。

(a) D1 四档文本（4 KB × 5 seed）

文本档	热启动 (bits/byte)	空白 (bits/byte)	gain（空白 − 热启动）
同分布 held-out	1.323 ± 0.239	—	$+5.061 \pm 0.580$
跨域（sft 尾部）	1.681 ± 0.081	—	$+4.373 \pm 0.912$
同分布但字节打乱	11.901 ± 0.463	—	-0.314 ± 0.331
零信息（全空格）	0.236	—	+2.688

(b) D2 / D2b 成员推断

条件	消费区 (bits/byte)	未消费 (bits/byte)	gap（消费 − 未消费）
热启动（权重 + 状态）	0.973 ± 0.173	1.323 ± 0.239	-0.350 ± 0.200
空白（仅权重）	—	—	$+0.365 \pm 0.828$

状态级样本记忆 = 热启动 gap − 空白 gap = $-\mathbf{0.714}$ bits/byte

- **组件粒度可到下界：**把「跨 patch 写入」这一整块干预拆成四个分量版本后，归因结论直接落在分量上（§5.5）——这在 RNN 的隐状态上看不到，因为隐状态没有暴露「按分量置换」的接口。

代价与纪律：状态跑一次就变，任何复用都会造成实验间污染；因此每次评测必须先克隆，且快照必须落盘（本架构的状态一旦被改写无法复原）。

7.3 与其他实验线的关系与展望

7.3.1 状态里装的是什么（D 线）

A1 说明热启动状态很值钱（3.73 bit/byte），D 线回答它值钱在哪里（表 10）。四个文本档上的 gain（空白 − 热启动）为：**同分布** $+5.061 \pm 0.580$、**跨域**（sft 尾部）$+4.373 \pm 0.912$、**同分布但字节打乱** -0.314 ± 0.331、**零信息**（全空格）+2.688（图 14）。解读：热启动记忆的主体是**通用序列先验**（跨域仍保留 ∼ 4.4 bit/byte），叠加少量**同分布偏向**（∼ 0.7 bit/byte）；对打乱字节没有先验，说明记忆锚定的是自然语言的结构而不是字节取值本身。

成员推断（D2 / D2b）是必须公开的一条。训练消费区文本在热启动下的 NLL 比未消费同分布文本低 -0.350 ± 0.200 bit/byte；把起点换成空白后这个差距消失（$+0.365 \pm 0.828$，与 0 无显著差异），反推**状态级样本记忆** = −0.714 bit/byte。也就是说：*逐样本的训练痕迹几乎完全由状态承载，权重没有贡献*。两个含义：其一，评测协议上任何用热启动的数字都带着「这几段文本模型可能见过」的脚注，主表必须双报起点状态（§4.3）；其二，隐私与去污染上，状态本身是记忆载体，这不只是统计压缩问题。

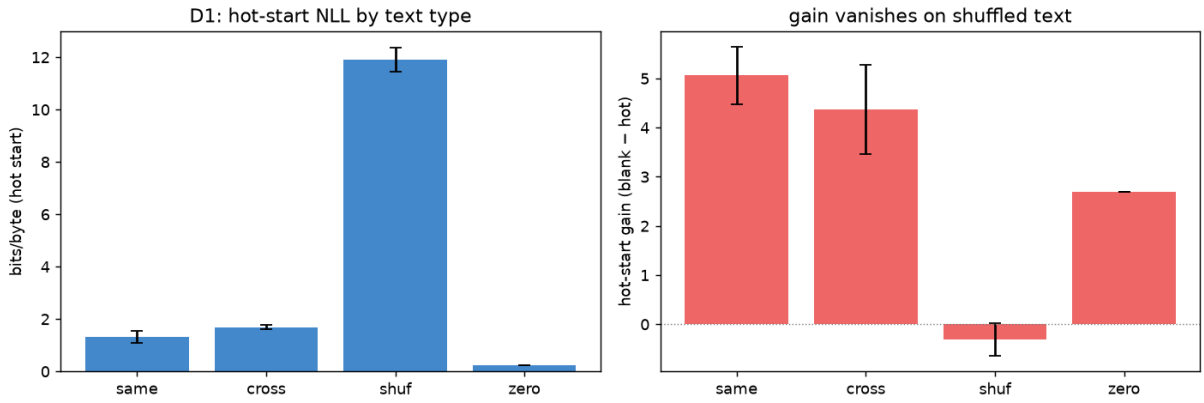


图 14: D1：热启动记忆的领域分解（4 KB × 5 seed）。左：各档文本的热启动 NLL；右：gain（空白 − 热启动）——跨域仍保留大部分收益，打乱字节的收益为负。

7.3.2 超长上下文的数值安全（E 线）

96 KB 单流：$\|\texttt{memory}\|_F \times 1.018$、$\|\texttt{MTM}\|_F \times 1.019$，`recurrent/queue` 范数构造性恒定；512 KB 单流：×1.132 与 ×1.065（图 9 左、图 13 左）。全程有界缓增、未见发散——这是 delta 规则 + 1/eff_seq_len 门控在数值上的正面证据。

污染与自愈（E2）：正常 8 KB + 随机字节 4 KB + 正常 8 KB 的流上，污染段内的 NLL 冲击为 +9.25（照常写入）与 +9.92 nat（污染段设为只读）；但**两组条件的尾部 NLL 只差** 0.0001（0.8608 vs 0.8607），残留都是 +0.085 nat。结论有两层：污染写入只造成**瞬时冲击**（污染后头 4 个 patch：0.686 vs 0.436 nat），**没有长期残留**；也因此「只读模式可作污染防御」这一说法**得不到数据支持**——残留量在两个条件下相同，尾部抬升更可能来自文本难度变化（与 §6.1 的漂移解释一致）。图见附录 D.2。

7.3.3 前向学习：连续适应而非离散绑定（G 线）

持续状态的另一面是「读着读着就变了」。合成任务上的结果（附录表 13、图 17、18）可以概括为一句话：**这个检查点上的前向学习是连续适应，而不是离散绑定**。（i）随机 ASCII 键值对联想：k 次演示后 write 条件仍在 8.3–8.6 bits（机会线 8.23），且不随 k 累积；write 与 *lock* 的差只有 +0.37–+0.66 bit/byte。（ii）把符号换成分布内高频词对后曲线**完全变平**（5.0–5.2 bits，与首字节机会线重合），说明这**不是**合成数据 OOD 造成的——是 delta 规则通道对离散配对的绑定本身很弱。（iii）自然语料上的 8-gram 复现增益只有 +0.054 nat，且在 *lock* 下消失。把这三条与 A2/A6 的 +1.00 bit/byte 放在一起看，**跨 patch 的收益主要来自连续的话题/句法适应，而不是「记住若干条事实」**。这既是本文的机制结论，也是把 needle 类精确检索降级为限定 claim 的直接理由（§6.4）。

7.3.4 下一步：训练侧实验（F 线）

本文的全部证据都指向同一个未解问题：为什么有效记忆只有 1–4k token？是**机制上界**（固定状态在这个写入规则下就只能记这么久），还是**训练进度**（该检查点只训到 20%，长程写入尚未学出来）？这需要训练侧实验才能分辨，按优先级：

1. **BPTT 窗口扫描**（bptt = 1/4/8）：bptt = 1 时梯度只到 patch 内，长程信用**只能**经前向 delta 规则传递。它的长程表现保留多少，即「前向通道」的贡献上限；1 → 4 的增益即梯度通道的贡献。这是本文「持续学习」主张在训练侧的判决性实验。

2. **永续 vs 文档边界重置训练**：直接检验「训练时不清状态到底是收益还是负担」。

3. **状态收敛（F5）**：用早期/中期/晚期检查点比较热启动收益与状态间相似度，回答「热启动之所以有效」的前提是否稳定。

4. **chunk_size（并行流数）扫描**：v_0d3 中 `memory` 的写入是跨流平均的，流数直接改变「每流写入占比」；v_0d4 改为序列级私有槽位后，流数变为「独立记忆实例数」，该扫描在两版语义下问的是不同的问题——与推理 batch= 1 的匹配度直接相关。

在这些实验完成之前，本文的结论应当被理解为**该检查点在该规模上的实测上界**，而不是该架构的能力上界。

8 局限与结论

8.1 局限

（1）单一检查点、单一规模，且只训练到 20%。本文全部数字来自 `gridman_medium_v_0d3_pretrain`（437.8M 参数，step 144,000/705,600），没有验证集、没有同预算基线（§6.3 未完成）。因此：绝对值（如 1.08 bit/byte）**不可**与任何外部模型对照，所有结论只在**条件间**成立；「有效记忆 1–4k token」也**不能**排除「换个训练更充分的检查点会显著更长」的可能。加验证集、训完 medium、补一个同预算的 byte Transformer 基线，是本稿升级为投稿稿的最低条件。

（2）精确检索与离散绑定。合成键值对与词对联想上，本检查点的绑定能力弱到接近机会线（§7.3.3）；据此本文把 needle 类精确检索降级为限定 claim，并如实说明「统计/语义级的长程记忆」才是本文证据支持的范围。B4 预实验本身**尚未运行**。

（3）热启动与训练语料的污染边界。状态级样本记忆 $= -0.714$ bit/byte 说明热启动起点含着训练语料的具体痕迹（§7.3.1）。本文据此要求主表双报起点状态；但「双报」只解决报告透明，不解决**污染本身**——严格版本需要用完全无重叠的评测集重做 A/B/C 三组。n-gram 重叠率的精确统计 [待补：待补，见附录 C]。

（4）受控遗忘曲线缺失。计划中的 C 线（注入键值对、按距离与组件分解测 recall 衰减）因实现缺陷作废（附录 D.1），修订版尚未重跑。这意味着本文**没有**一条真正受控的遗忘曲线：§6.2 的重复距离是对自然文本的观察性测量，其各距离桶之间并非完全受控（不同距离上重复的是不同 n-gram 群体，首次出现难度本就不同）。

（5）实现版本已演进，部分归因结论绑定旧语义。本文数字全部出自 v_0d3 检查点；现行 v_0d4 把 `memory` 改为序列级私有槽位（无跨流平均、无跨卡同步）。推理侧（batch $= 1$）两版逐位对应，主结论（S1/S2、A1/A2/B2）不受影响；但**训练侧语义已变**：v_0d3 的「重置 memory $=$ 动共享知识」在 v_0d4 中退化为只影响单流，§5.5 的组件归因、A5 的 memory 回滚语义在 v_0d4 检查点上需重新验证。此外 v_0d3/v_0d4 存档互不兼容，跨版续训不可行。

（6）未完成项。A4（状态错配）、B3（等效滑窗基线）、B4（needle 预实验）、F 线（训练侧全部实验）、E3（精度链漂移）、E4（自由运行分叉）均未运行，正文与附录中一律标注为待补，不以任何形式给出推测数值。

（7）口径订正史（主动披露）。本稿在定稿前对实验包做了逐脚本审读，修正了三处口径问题（n-gram 分桶缺陷导致的一条错误结论已撤回、A1 计分窗口未对齐、C1 答案位置偏移导致该实验作废），并补了模型无关的回归自检。保留这段历史的原因是：其中一条（「长程零衰减」）曾经看起来非常漂亮，而它恰恰来自一个会让所有桶报告同一份数据的实现缺陷。**分桶/配对类统计必须断言样本数互不相同**，这条纪律比结论本身更重要。

8.2 结论

本文把「固定大小的持续状态能承载多少跨上下文信息」当作一个可测量的经验问题，在一个 437.8M 参数的 byte 级有状态语言模型上，用 teacher-forced bits/byte 与状态手术（热启动/空白、commit/*lock*、分量重置/冻结）做了系统消融，得到三条可以直接引用的结论。

第一，状态确实在携带信息，且写入有因果贡献。同一计分窗口下热启动比空白低 3.73 bit/byte 且无收敛趋势；锁定跨 patch 写入的代价是 $+1.00 \pm 0.09$ bit/byte，并随上下文累积。S1 成立。

第二，承载者是块级循环状态，不是参数量最大的矩阵记忆。冻结 `recurrent` 的代价（+1.022 bit/byte）与冻结全部写入（+1.034 bit/byte）几乎相同；冻结 delta 规则矩阵记忆的代价为零（−0.0005 bit/byte），但去掉热启动使损失升高 3.7 bit/byte 以上——**矩阵记忆的价值在读取训练稳态先验，不在增量适应**。这个结论由 A2/A5/A6/B2-freeze 四组独立实验交叉支撑。

第三，有效记忆距离约为 1–4k token，且长度不是瓶颈。自然重复的复现增益集中在 <1k（+0.084 nat），4k 之外衰减到零；同时 512 KB 连续写入下状态范数有界（memory ×1.13），重置频率扫描单调无 U 形。因此「永续写入」在数值上是安全的，瓶颈在容量与写入/读出机制，而不是「能不能一直写下去」。

对「永续上下文」主张的净判断：固定大小状态**可以**持续携带信息，并且**可以**在远长于其有效记忆距离的流上稳定运行；但在本文这个检查点上，它的记忆能力等效于数 k token 的滑动记忆，尚不足以支撑「记住了整条流」的强主张。下一步的判决性实验是 BPTT 窗口扫描（bptt = 1 保留多少长程能力）与永续 vs 边界重置训练的对照（§7.3.4）——前者区分「前向通道上界」与「梯度通道贡献」，后者直接回答「训练时不清状态是收益还是负担」。

A 完整实验配置、检查点与复现

表 11：配置与检查点事实。标「推算」的量为本文推算，其余引自团队实现文档与配置。

项	值	备注
模型配置	`GRIDMAN_MEDIUM`：`embed_dim`=1024，`blocks`=3，`patch_size`=64（本文检查点 v_0d3：`chunk_size`=192；现行 v_0d4 MEDIUM：`chunk_size`=1，`bptt`=16）	4 个 `c_state`
实现版本	行为描述 v_0d4（2026-10-18）；全部数字出自 v_0d3 检查点，存档不互相兼容	见 §3 表 1
参数量	437.8M	权重 + 状态合计
词表	300（字节 0–255，`pad`=256，`eos`=257）	byte 级，无 OOV
状态规模	batch= 1 切片合计 14,072,832 个 fp32 元素（≈ 53.7 MiB）：`memory` 12,954,624（49.4 MiB）+ `MTM` 1,048,576（4.0 MiB）+ `queue` 65,536（0.25 MiB）+ `recurrent` 4,096（0.016 MiB）	按 v_0d4 `StateLayout` 实测
训练	AdamW + cosine，clip 1.0，bf16，lr 5×10^{-4}，`bptt`=4 patch（256 token）	192 条并行永续流（v_0d3 检查点口径）
检查点	`gridman_medium_v_0d3_pretrain`，step 144,000/705,600（20%）	唯一使用的权重
数据		
预训练	`pretrain.jsonl` ≈ 9.5 GB，每行 `{"text": ...}`	每条流 chunk ≈ 49.4 MB
微调	`sft.jsonl` ≈ 14 GB，多轮对话（仅 assistant token 计 loss）	用作跨域档
已消费区	每条流前 144,000 × 64 B = 9.2 MB	推算
held-out 取行	每条流 chunk 内偏移 ≥ 20 MB	零污染
复现：脚本 → 实验		
`common_eval.py`	脚手架：模型单例、teacher-forced 流式评测、文本源、组件手术	全部实验共用

续下页

表 11 （续）

项	值	备注
a1_hot_vs_blank.py	A1（§5.1）	后者为对齐窗口版
/		
a1_clean_heldout.py		
stage1_a2_a3_a5.py	A2、A3、A5（§5.2、§5.3、§5.5）	
stage1b_a6.py	A6 组件冻结（§5.5）	
stage2_d1_d2.py	D1、D2、D2b（附录 E）	2b 为 5-seed 修订版
/		
stage2b_d1_d2.py		
/		
stage8_d2b.py		
stage3_e1_e5_b2.py	E1、E5、B2（§6.1、§6.2）	含 512 KB 长流
/		
stage6_g1c_e1long.py		
stage4_g1_g2.py	G1a、G1b、G2（附录 E）	
stage5_c1_e2.py	C1（作废，附录 D.1）与 E2（附录 D.2）	
stage7_b2_freeze.py	B2-freeze 组件归属（§6.2）	
make_figures.py	全部图表	
/		
make_fig_a1.py		
verify_positions.py	模型无关回归自检（答案位置必须落在自己的 kv 块内）	无 GPU 也可跑

B 逐位置完整曲线

正文引用的逐位置曲线：图 5（A1 双曲线 + 差值）、图 6（A2 写入 vs 只读）、图 7（跨文档边界对齐恢复曲线）、图 9（96 KB 范数 + per-position）、图 10（重置频率）、图 13（512 KB 范数 + 距离分桶）、图 11 与图 12（重复距离与组件归属）。附录 D.2 给出污染自愈曲线（图 16）。

C held-out 去污染统计

本文的 held-out 切分按「每条流 chunk 内偏移 ≥ 20 MB」取行（§4.1），该偏移由「step $\times$ patch 长度 $= 9.2$ MB」推算得到，留有 $> 2\times$ 余量。B2 的分桶统计顺带产出了配对数量（表 8），可作为分布性的旁证。[待补：待补：与训练集（已消费区）的 4-gram 重叠率、以及 n-gram 去重后的行级去重率明细；口径与实现见《论文大纲-B》第 1 章。]

D C 线与 E 线：遗忘、污染与稳定性

D.1 C1 受控遗忘曲线：作废说明

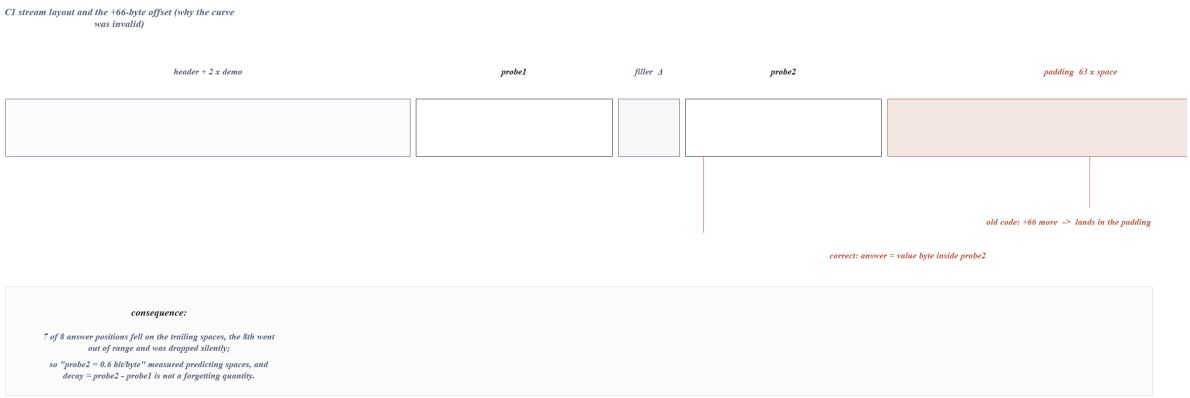


图 15: C1 的流结构与那 66 字节的偏移（Visio 绘制，源文件 `figs/fig_c1_stream.vsdx`）。段宽按字节数等比（filler 段压缩），红框为末尾补白；两条竖线分别是「正确答案位置」与「旧代码算出的位置」。

实验意图。构造 `[header + 2×demo + probe1] + filler(Δ)`（图 15）+ `probe2` 的合成流（8 对随机 ASCII 键值对、2 次演示），在 $\Delta \in \{512, 2048, 8192, 32768, 131072\}$、4 个条件（正常 / filler 期只读 / filler 期清零 `recurrent` / filler 期清零 `queue+MTM`）、3 个 seed 下测「probe2 相对 probe1 的 recall 衰减」，期望得到按组件分解的遗忘曲线。

实测异常。probe1 的答案位 NLL = 8.38 bits（机会线 8.23），而 probe2 = 0.6 bits——注入后立刻测不会，隔着最多 128K *token* 反而几乎白送；四个条件的衰减全挤在 −7.5–−7.8，彼此不可分。

根因（已定位）。生成脚本把块内偏移基准 $D = \text{header} + 2 \times \text{demo} = 66$ 字节用在了 probe2 上，而 probe2 的位置又已经叠加了「已注入长度 + Δ」，于是 **66 字节被算了两次**：8 个答案位置里 7 个落到 probe2 块之后的**补白空格**上、第 8 个越界被静默丢弃（代码在位置不在流内时只是跳过）。所以「probe2 = 0.6 bits」的真实含义是「模型很会猜一串连续空格」，而 decay = probe2 − probe1 是伪量。

处理。该实验作废，本文正文不引用其任何数字；修订版已实现（把「块 → 答案位置」的算术抽成公共函数，位置一律由「块在流中的绝对起点」推出，并加断言：答案位置的前一字节必须是 `=`、后一字节必须是 `|`），配套的模型无关自检脚本可在无 GPU 环境下验证位置正确性（旧实现会当场失败）。**重跑尚未进行**（预计与原实现同量级，约 80 分钟）。即使重跑，也需要先注意一个先行事实：本检查点对随机 ASCII 键值对的绑定本身很弱（附录 E 的 G1a：8.3–8.6 bits ≈ 机会线），因此该实验更可能给出「记不住」而不是「记得住但会忘」的结果——正确的写法是把这类结论归到「离散绑定弱」的机制结论上，而不是包装成遗忘曲线。

D.2 E2 状态污染与自愈

流结构：正常 8 KB + 随机字节 4 KB + 正常 8 KB；两条件为 `write_through`（污染段照常写入）与 `lock_garbage`（污染段只读）。结果见表 12 与图 16：污染段内冲击巨大（+9.25 / +9.92 nat），但两组条件的尾值与残留几乎相同（0.8608 vs 0.8607；+0.085 vs +0.085 nat），污

染后前 4 个 patch 才有明显差异（0.686 vs 0.436 nat）。**结论：**污染写入造成瞬时冲击、无长期残留；「只读模式作为污染防御」得不到数据支持。尾部相对基线的 +0.085 nat 抬升更可能来自文本难度漂移（与 §6.1 的 E5 判据一致）。

表 12: E2 污染与自愈（nats）。恢复长度按「回到基线 +0.5 以内」判定。

条件	污染前	污染段内	污染后 4 patch	尾部	残留	恢复（patch）
write_through	0.776	10.027	0.686	0.861	+0.085	0
lock_garbage	0.776	10.696	0.436	0.861	+0.085	0

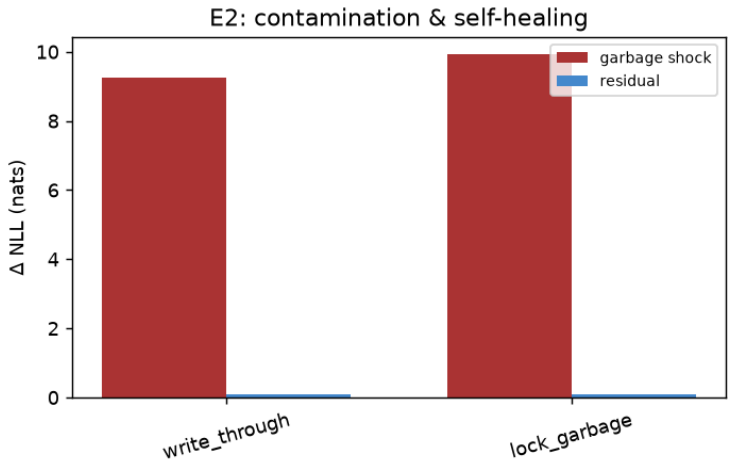


图 16: E2：污染段的冲击与残留。冲击量级远大于残留，且两条件的残留相同。

E　D 线与 G 线的完整数字

表 13: G 线前向学习：合成任务与自然语料。指标为答案位 NLL 的均值（bits），5 个 seed 的 95% CI；write / lock 两条件。数值越大越差，机会线见表内注。

任务	设定	write	lock
G1a 随机 ASCII 键值对联想	$k = 0$（无演示）	8.93 ± 0.62	8.93 ± 0.62
（机会线 $\log_2 85 \approx 6.4$ bits）	$k = 1$	7.54 ± 0.49	7.54 ± 0.49
	$k = 2$	8.31 ± 0.34	8.98 ± 0.65
	$k = 8$	8.53 ± 0.61	8.98 ± 0.65
	$k = 16$	8.61 ± 0.72	8.98 ± 0.65
G1b 单字节 bigram 转移	$k \le 8$	≈ 3.6–3.9	同 write
（CI 很大，仅定性）	$k = 16$	3.15 ± 1.82	3.91 ± 2.21
	$k = 32$	3.33 ± 1.96	3.91 ± 2.21
G1c 分布内高频词对联想	$k = 0$	5.15 ± 0.31	5.34 ± 0.58
	$k = 8$	4.96 ± 0.14	5.17 ± 0.30
	$k = 16$	5.19 ± 0.12	5.24 ± 0.43
G2 自然语料 8-gram 复现	首次 / 二次（nats）	$0.732 \to 0.678$	$1.483 \to 1.499$
（32 KB，3382 对，间距 $<$4k）	增益 (nats)	$+\mathbf{0.054}$	-0.016

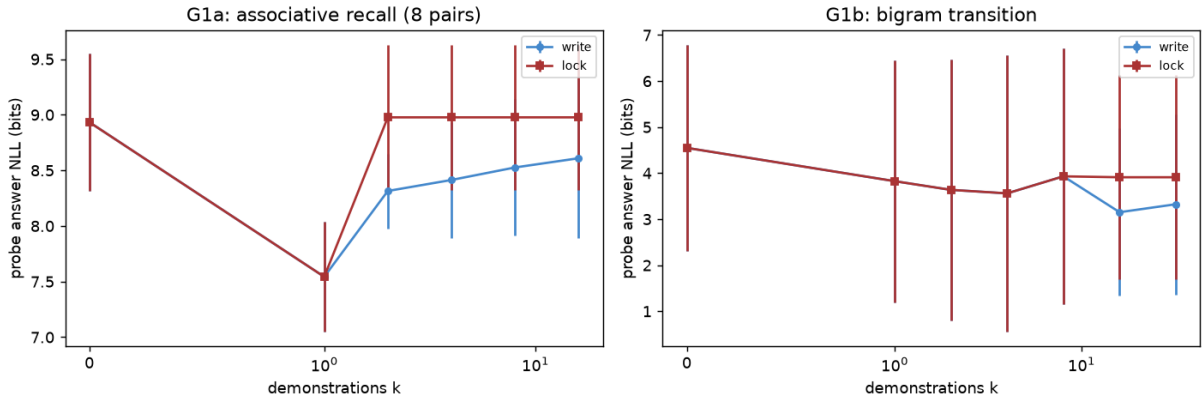


图 17: G1a / G1b：前向学习曲线（k 为演示次数）。联想任务上 write 曲线不随 k 累积；bigram 任务需要 $k \ge 16$ 才出现跨 patch 效应，且置信区间极大。

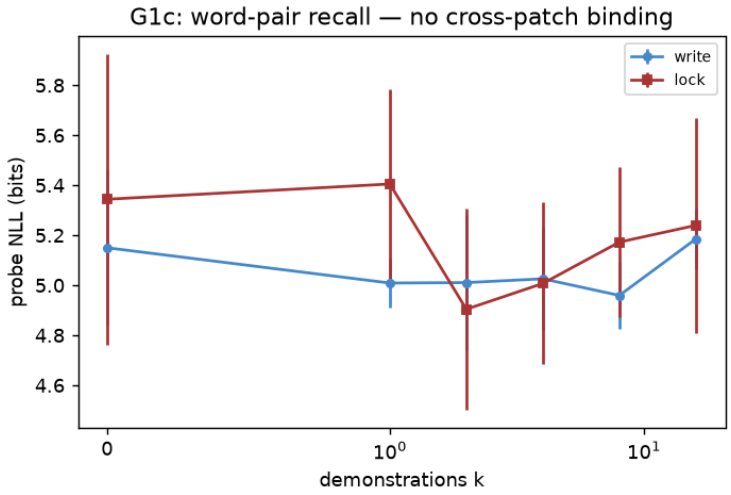


图 18: G1c：把符号换成分布内高频词对后曲线依然全平（≈ 首字节机会线），排除「随机 ASCII 太 OOD」这一解释。

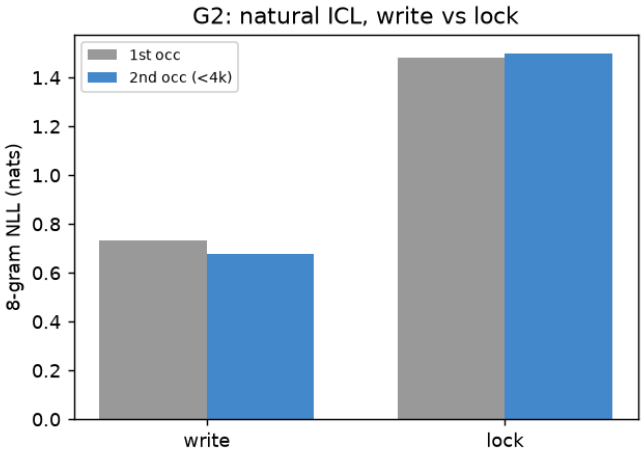


图 19: G2：自然语料 8-gram 的二次出现增益 +0.054 nat，且在 *lock* 下消失——说明该增益是写入依赖的、且量级很小。

F　F 线与 H 线：未运行

F 线（训练动力学）需要重训，超出「只用现成权重」的范围：BPTT 窗口扫描、永续 vs 文档边界重置、状态统计演化、`chunk_size` 扫描、状态收敛时间。H 线（任务级 demo：多会话一致性、长文档跨块一致性、代码跨文件一致性）依赖推理链冻结后再做。两项都按 §7.3.4 的优先级排序，本稿不给任何推测数值。

参考文献

[1] Jimmy Ba, Geoffrey E. Hinton, Volodymyr Mnih, Joel Z. Leibo, and Catalin Ionescu. Using fast weights to attend to the recent past. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2016.

[2] Ali Behrouz, Peilin Zhong, and Vahab Mirrokni. Titans: Learning to memorize at test time. *arXiv preprint arXiv:2501.00663*, 2025.

[3] Iz Beltagy, Matthew E. Peters, and Arman Cohan. Longformer: The long-document transformer. 2020.

[4] Tom B. Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared Kaplan, et al. Language models are few-shot learners. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2020.

[5] Aydar Bulatov, Yuri Kuratov, and Mikhail S. Burtsev. Recurrent memory transformer. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2022.

[6] Stephanie C. Y. Chan, Adam Santoro, Andrew K. Lampinen, Jane X. Wang, Aaditya Singh, Pierre H. Richemond, James McClelland, and Felix Hill. Data distributional properties drive emergent in-context learning in transformers. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2022.

[7] Kyunghyun Cho, Bart van Merriënboer, Caglar Gulcehre, Dzmitry Bahdanau, Fethi Bougares, Holger Schwenk, and Yoshua Bengio. Learning phrase representations using rnn encoder–decoder for statistical machine translation. In *Proceedings of EMNLP*, 2014.

[8] Tri Dao, Daniel Y. Fu, Stefano Ermon, Atri Rudra, and Christopher Ré. Flashattention: Fast and memory-efficient exact attention with io-awareness. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2022.

[9] Jiayu Ding, Shuming Ma, Li Dong, Xingxing Zhang, Shaohan Huang, Wenhui Wang, and Furu Wei. Longnet: Scaling transformers to 1,000,000,000 tokens. *arXiv preprint arXiv:2307.02486*, 2023.

[10] Shibhansh Dohare, J. Fernando Hernandez-Garcia, Qingfeng Lan, Parash Rahman, A. Rupam Mahmood, and Richard S. Sutton. Loss of plasticity in deep continual learning. *Nature*, 632:768–774, 2024.

[11] Edouard Grave, Armand Joulin, and Nicolas Usunier. Improving neural language models with a continuous cache. In *International Conference on Learning Representations (ICLR)*, 2017.

[12] Albert Gu and Tri Dao. Mamba: Linear-time sequence modeling with selective state spaces. *arXiv preprint arXiv:2312.00752*, 2023.

[13] Sepp Hochreiter and Jürgen Schmidhuber. Long short-term memory. volume 9, pages 1735–1780, 1997.

[14] Gregory Kamradt. Needle in a haystack: Pressure testing llms. GitHub repository, https://github.com/gkamradt/LLMTest_NeedleInAHaystack, 2023.

[15] Jared Kaplan, Sam McCandlish, Tom Henighan, Tom B. Brown, Benjamin Chess, Rewon Child, Scott Gray, Alec Radford, Jeffrey Wu, and Dario Amodei. Scaling laws for neural language models. *arXiv preprint arXiv:2001.08361*, 2020.

[16] Angelos Katharopoulos, Apoorv Vyas, Nikolaos Pappas, and François Fleuret. Transformers are rnns: Fast autoregressive transformers with linear attention. In *International Conference on Machine Learning (ICML)*, 2020.

[17] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe Kiela. Retrieval-augmented generation for knowledge-intensive nlp tasks. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2020.

[18] Catherine Olsson, Nelson Elhage, Neel Nanda, Nicholas Joseph, Nova DasSarma, Tom Henighan, et al. In-context learning and induction heads. 2022.

[19] Bo Peng, Eric Alcaide, Quentin Anthony, Alon Albalak, Samuel Arcadinho, et al. Rwkv: Reinventing rnns for the transformer era. In *Findings of the Association for Computational Linguistics: EMNLP*, 2023.

[20] Reiner Pope, Sholto Douglas, Aakanksha Chowdhery, Jacob Devlin, James Bradbury, Jonathan Heek, Kefan Xiao, Shivani Agrawal, and Jeff Dean. Efficiently scaling transformer inference. *Proceedings of Machine Learning and Systems (MLSys)*, 2023.

[21] Imanol Schlag, Kazuki Irie, and Jürgen Schmidhuber. Linear transformers are secretly fast weight programmers. In *International Conference on Machine Learning (ICML)*, 2021.

[22] Jürgen Schmidhuber. Learning to control fast-weight memories: An alternative to dynamic recurrent networks. *Neural Computation*, 4(1):131–139, 1992.

[23] Yu Sun, Xinhao Li, Karan Dalal, Jiacheng Xu, Arjun Vikram, Genghan Zhang, Yizhe Tian, Xinlei Chen, Xiaolong Wang, et al. Learning to (learn at test time): Rnns with expressive hidden states. *arXiv preprint arXiv:2407.04620*, 2024.

[24] Yutao Sun, Li Dong, Shaohan Huang, Shuming Ma, Yuqing Xia, Jilong Xue, Jianyong Wang, and Furu Wei. Retentive network: A successor to transformer for large language models. *arXiv preprint arXiv:2307.08621*, 2023.

[25] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2017.

[26] Songlin Yang, Bailin Wang, Yikang Shen, Rameswar Panda, and Yoon Kim. Parallelizing linear transformers with the delta rule over sequence length. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2024.